



**Politechnika
Śląska**

PRACA MAGISTERSKA

System przeznaczony dla osób niewidomych do rozpoznawania dotykanych
obiektów 3D

Piotr MARCOL

Nr albumu: 300463

Kierunek: Informatyka

Specjalność: Oprogramowanie Systemowe

PROWADZĄCY PRACĘ

Dr hab. inż., prof. PŚ Michał Maćkowski

KATEDRA Systemów Rozproszonych i Urządzeń Informatyki

Wydział Automatyki, Elektroniki i Informatyki

Gliwice 2026

DRAFT

Tytuł pracy

System przeznaczony dla osób niewidomych do rozpoznawania dotykanych obiektów 3D

Streszczenie

(Streszczenie pracy – odpowiednie pole w systemie APD powinno zawierać kopię tego streszczenia.)

Słowa kluczowe

(2-5 słów (fraz) kluczowych, oddzielonych przecinkami)

Thesis title

A system designed for the blind to recognize touched 3D objects

Abstract

(Thesis abstract – to be copied into an appropriate field during an electronic submission – in English.)

Key words

(2-5 keywords, separated by commas)

DRAFT

Spis treści

1	Wstęp	1
1.1	Motywacja podjęcia tematu	1
1.2	Cel i zakres pracy	1
1.3	Główne założenia i ograniczenia	1
1.4	Wkład autora	2
1.5	Struktura pracy	2
2	Analiza problemu i przegląd literatury	3
2.1	Perspektywa osób niewidomych w rozpoznawaniu obiektów przestrzennych	3
2.2	Technologie wspomagające osoby z niepełnosprawnością wzroku	6
2.2.1	Rozwój technologii asystujących	6
2.2.2	Standardy i regulacje prawne dotyczące dostępności	6
2.2.3	Współczesne systemy wspomagające osoby niewidome	7
2.3	Problem rozpoznawania obiektów 3D w interakcji dotykowej	9
2.4	Metody detekcji obiektów w obrazach	11
2.4.1	Sformułowanie problemu detekcji obiektów	11
2.4.2	Klasyczne metody wizji komputerowej	13
2.4.3	Detektory dwuetapowe	13
2.4.4	Detektory jednoetapowe	14
2.4.5	Detektory oparte na architekturze transformera	15
2.4.6	Porównanie metod detekcji	15
2.5	Metryki oceny detekcji obiektów	17
2.6	Rodzina modeli YOLO	19
2.6.1	Architektura modeli YOLO	19
2.6.2	Warianty ze względu na realizowane zadanie	22
2.6.3	Warianty ze względu na skalę modelu	23
2.6.4	Dobór wariantów modelu	24
2.7	Wykorzystanie danych wizualnych i przestrzennych w rozpoznawaniu obiektów	24
2.7.1	Obraz RGB	25

2.7.2	Pozyskiwanie informacji o głębi	26
2.7.3	Dane LiDAR i chmury punktów	27
2.7.4	Fuzja danych wizualnych i przestrzennych	28
2.8	Podsumowanie analizy tematu	29
3	Koncepcja systemu rozpoznawania dotykanych obiektów 3D	31
3.1	Założenia funkcjonalne i нефункционалне	31
3.2	Architektura systemu	33
3.3	Dobór klas rozpoznawanych obiektów	34
3.4	Uzasadnienie wyboru metody detekcji	36
3.5	Przepływ danych w systemie	36
3.6	Ograniczenia przyjętej koncepcji	36
4	Implementacja systemu	37
4.1	Środowisko programistyczne i wykorzystane narzędzia	37
4.2	Moduł akwizycji danych obrazowych	37
4.3	Moduł komunikacji klient-serwer	37
4.4	Moduł detekcji obiektów	37
4.5	Moduł informacji zwrotnej dla użytkownika	37
4.6	Organizacja kodu i plików projektu	38
5	Zbiór danych i metodyka badań	39
5.1	Charakterystyka zbioru danych	39
5.2	Proces pozyskiwania i oznaczania danych	39
5.3	Podział danych na zbiory treningowy, walidacyjny i testowy	39
5.4	Warianty danych wejściowych	39
5.5	Konfiguracja treningu modeli	39
5.6	Metryki oceny	40
5.7	Procedura eksperymentalna	40
6	Wyniki badań i ich analiza	41
6.1	Wyniki uczenia modeli YOLO	41
6.2	Porównanie wariantów modeli	41
6.3	Wpływ reprezentacji obrazu na skuteczność detekcji	41
6.4	Analiza błędów klasyfikacji i detekcji	41
6.5	Ocena działania systemu w scenariuszu użytkowym	41
6.6	Dyskusja ograniczeń wyników	42
7	Podsumowanie i wnioski końcowe	43
7.1	Podsumowanie wykonanych prac	43
7.2	Ocena realizacji celu pracy	43

7.3	Najważniejsze wnioski	43
7.4	Możliwe kierunki rozwoju	43
Bibliografia		51
Dokumentacja techniczna		55
Spis skrótów i symboli		57
Lista dodatkowych plików, uzupełniających tekst pracy (jeżeli dotyczy)		61
Spis rysunków		63
Spis tabel		65

DRAFT

Rozdział 1

Wstęp

To powinien być rozdział „ustawiający” całą pracę. Powinny się tu znaleźć:

- kontekst: technologie wspomagające osoby z niepełnosprawnością wzroku,
- problem: rozpoznawanie dotykanych obiektów 3D w czasie zbliżonym do rzeczywistego,
- cel pracy,
- zakres pracy,
- wkład własny,
- krótka charakterystyka rozdziałów.

1.1 Motywacja podjęcia tematu

Tu piszę, dlaczego temat ma sens: samodzielność osób niewidomych, rozpoznawanie obiektów, ograniczenia obecnych rozwiązań.

1.2 Cel i zakres pracy

Cel np.: „Celem pracy jest opracowanie i eksperymentalna ocena systemu wspomagającego rozpoznawanie dotykanych obiektów 3D z wykorzystaniem metod detekcji obiektów na obrazie.”

1.3 Główne założenia i ograniczenia

Tu można wpisać: iPhone jako urządzenie wejściowe, obiekty drukowane 3D, sześć klas, detekcja w obrazie, komunikacja z serwerem, informacja zwrotna audio.

1.4 Wkład autora

Bardzo ważne, bo wymagania tego chcą. Wkład: przygotowanie datasetu, implementacja klienta/serwera, trening modeli, eksperymenty, analiza wyników.

1.5 Struktura pracy

Krótki opis, co jest w każdym rozdziale.

DRAFT

Rozdział 2

Analiza problemu i przegląd literatury

Zagadnienie rozpoznawania obiektów przez osoby niewidome wymaga uwzględnienia nie tylko aspektów technicznych, ale też społecznych i psychologicznych – tego, jak użytkownik poznaje otoczenie i jakie informacje z niego wyciąga. Osoby widzące zazwyczaj nie mają problemów z rozpoznawaniem kształtów, kolorów, orientacji czy położenia obiektów w przestrzeni, natomiast osoby niewidome i słabowidzące w większym stopniu wykorzystują inne zmysły i technologie wspomagające, aby móc zbudować mentalną reprezentację otoczenia.

Ze względu na to, projektowanie i wykonanie systemu rozpoznawania obiektów nie może skupiać się wyłącznie na aspektach technicznych, ale należy również uwzględnić perspektywę użytkownika docelowego. Należy zrozumieć, w jaki sposób informacja przekazywana przez komputer może wesprzeć proces poznawania otoczenia. W niniejszym rozdziale przedstawiono perspektywę osób niewidomych w kontekście rozpoznawania obiektów przestrzennych, a następnie omówiono wybrane technologie wspomagające osoby z niepełnosprawnością wzroku, metody komputerowej detekcji obiektów, metryki oceny modeli i możliwość wykorzystania zróżnicowanych danych wejściowych, takich jak obrazy RGB, mapy głębi czy dane LiDAR. Szczególną uwagę poświęcono rodzinie modeli YOLO, wykorzystywanej w niniejszej pracy. Na koniec podsumowano przeprowadzoną analizę.

2.1 Perspektywa osób niewidomych w rozpoznawaniu obiektów przestrzennych

Przy projektowaniu systemu przeznaczonego dla osób niewidomych należy wziąć pod uwagę różnice w percepcji przestrzennej pomiędzy osobami widzącymi a niewidomymi. W literaturze zagadnienie to jest analizowane zarówno z perspektywy technicznej, psychologicznej, jak i pedagogicznej. Badania nad percepcją osób niewidomych pozwalają lepiej

zrozumieć ich sposób poznawania otoczenia. Costa et al. w swojej pracy, na podstawie studium przypadku, pokazują, że rozpoznawanie obiektu jest całym procesem stopniowego pozyskiwania i integrowania informacji. Zaznaczają, że w przypadku osób widzących spojrzenie zazwyczaj pozwala zrozumieć obiekt całościowo, natomiast w przypadku osób niewidomych poznanie odbywa się sekwencyjnie, etapami. Duże znaczenie mają więc cechy obiektu, które są wyraźnie dostępne w eksploracji dotykowej: prostota, regularność, krawędzie oraz faktura. Wspierają one budowę mentalnej reprezentacji przestrzennej obiektu. Osoba niewidoma może opierać ją nie tylko na aktualnej informacji dotykowej, ale również wcześniejszych doświadczeniach, kontekście i wiedzy o świecie. Autorzy ukazują, że brak wzroku nie oznacza prostego zastąpienia go przez inne zmysły – dotyk, słuch, pamięć i doświadczenia dostarczają informacji innego rodzaju oraz są wykorzystywane w odmienny sposób [5].

Rozwinięcie tej myśli znajdujemy w pracy autorstwa Figueiras i Arcavi, którzy pokazują jak osoba niewidoma uczy się matematyki i w jaki sposób komunikuje się z nauczycielem. Autorzy opisują przebieg lekcji prowadzonych przez niewidomego nauczyciela dla trzech uczniów, z czego dwóch było w pełni niewidomych, a jeden miał częściową utratę wzroku. Szczególnie istotne okazują się zapisy rozmów pomiędzy nauczycielem a uczniami, które ukazują wcześniej wspomniane etapowe poznawanie obiektu. Podczas omawiania stożka nauczyciel bazuje na wcześniejszych doświadczeniach i wiedzy uczniów, a także na ich wyobrażeniach przestrzennych.

W kolejnej części artykułu autorzy ukazują w jaki sposób nauczyciel prowadzi uczniów przez analizę wykresu funkcji kwadratowej. Nauczyciel celowo odracza rozwiązanie algebraiczne i prowadzi uczniów przez proces rozumowania. Początkowo uczestnicy rozważają położenie jednego z pierwiastków, następnie wierzchołka i osi symetrii. Dopiero po tym, gdy uczniowie przeprowadzili rozumowanie, nauczyciel pozwala na rozwiązanie algebraiczne. Pokazuje to, że w przypadku, gdy nie mamy dostępu do globalnej reprezentacji obrazu, uwypukla się znaczenie jawnych opisów relacji pomiędzy charakterystycznymi cechami obiektu, odwołania do wcześniejszych reprezentacji i uporządkowanie procesu rozumowania [13]. W kontekście projektowanego systemu sugeruje to, że sama informacja o klasie rozpoznanego obiektu nie jest wystarczająca, a zasadne może być również przekazywanie informacji o jego cechach charakterystycznych i zachodzących między nimi relacjach.

Odchodząc od ściśle teoretycznych i obserwacyjnych badań, w literaturze znajdujemy również przykłady artykułów naukowych, które bezpośrednio wskazują problemy osób niewidomych. Güler i Ürey przeprowadzili badanie jakościowe obejmujące 13 uczniów szkoły dla osób z niepełnosprawnością wzroku oraz ich 10 nauczycieli. Dotyczyło ono wyzwań i potrzeb uczniów przy nauce przedmiotów przyrodniczych. Istotnym wnioskiem z dokumentu jest wskazanie, że nauczyciele częściej korzystali z podręczników pisanych alfabetem Braille’a oraz tabliczek i ryłców do zapisu brajlowskiego, aniżeli z materiałów

przestrzennych, modeli 3D czy eksperymentów. Jeden z uczniów wskazywał, że ma problem z wyobrażaniem sobie reprezentacji przestrzennych słów, twierdząc, że gdy słyszy np. słowo "piramida" nie potrafi zbudować w umyśle odpowiadającego mu obrazu. Nauczyciele wskazują również, że tureccy uczniowie dotknięci niepełnosprawnością wzroku nie mają przewidzianego osobnego toku nauczania. Są więc zmuszeni do konkurowania z widzącymi rówieśnikami tak, jakby byli w stanie wyuczyć się wszystkiego w ten sam sposób. Zaznaczają też, że zagadnienia związane z naukami przyrodniczymi są często abstrakcyjne, co utrudnia ich zrozumienie, a sam proces nauki powinien być dostosowany do potrzeb uczniów z niepełnosprawnością wzroku [15]. Podobne problemy związane z brakiem materiałów edukacyjnych, koniecznością dostosowania istniejących, wydłużonym czasem przygotowania zajęć i dostosowania komunikacji odnotowano również w badaniach dotyczących nauki języka angielskiego [2].

Kolejnym zagadnieniem dotyczącym perspektywy osób niewidomych, na które zwrócili uwagę Li et al., jest sposób, w jaki różne źródła informacji – dotyk, opis dźwiękowy, wcześniejsze doświadczenia – wspierają rozumienie obiektów. Zaznaczają, że osoby niewidome nie bazują jedynie na pojedynczym źródle informacji, ale łączą informacje pochodzące z różnych modalności. Badana grupa osób wskazała, że poprzez dotyk mogą zrozumieć kształt obiektu, jego położenie, formę i szczegóły przestrzenne, ale by w pełni zrozumieć kontekst, historię lub znaczenie obiektu, potrzebują dodatkowego opisu audio. Dodatkowo, autorzy zwracają uwagę na to, że osoby niewidzące różnią się między sobą pod względem preferencji i sposobów poznawania obiektów. Zaznaczają, że różnią się one w zależności od stopnia niepełnosprawności wzroku oraz tego, czy osoba jest niewidoma od urodzenia, czy utraciła wzrok w późniejszym okresie życia [20]. Na tej podstawie można przyjąć, że projektowany system nie powinien ograniczać się do jednego sposobu komunikacji, ale pozwolić na personalizację i dostosowanie do preferencji konkretnego użytkownika.

Przytoczone badania wskazują, że rozpoznawanie obiektów przez osoby niewidome nie jest pojedynczym aktem identyfikacji, ale etapowym procesem poznawczym. Eksploracja dotykowa pozwala na poznanie lokalnych części obiektu: krawędzi, powierzchni, faktury i orientacji, ale to wcześniejsze doświadczenia i przekaz słowny pozwalają na zbudowanie jego pełnej reprezentacji mentalnej. Jednocześnie, niewielka liczba dedykowanych materiałów edukacyjnych mocno utrudnia procesy nauczania – zwłaszcza przy pojęciach abstrakcyjnych.

Wynika z tego, że system wspomagający nie powinien ograniczać się jedynie do przekazywania nazw obiektów, ale wspierać użytkownika w całym procesie jego rozpoznawania. Informacja zwrotna powinna być jednoznaczna, możliwa do dostosowania do potrzeb użytkownika i wspierać go w naturalnym procesie eksploracji dotykowej. Te wnioski są podstawą do analizy istniejących rozwiązań technologicznych przedstawionej w kolejnym podrozdziale.

2.2 Technologie wspomagające osoby z niepełnosprawnością wzroku

Analiza perspektywy osób niewidomych pozwala zauważyć, że odmienne sposoby poznawania przestrzeni wokół nich wiążą się z potrzebą wsparcia w codziennym funkcjonowaniu. Może ono przyjmować formę tradycyjnych rozwiązań, takich jak biała laska lub pies przewodnik, pomocy drugiej osoby, a także coraz bardziej zaawansowanej technologii asystującej, która ciągle rozwijana daje coraz lepsze możliwości.

2.2.1 Rozwój technologii asystujących

Naukowcy od dawna widzieli w rozwoju techniki okazję do wsparcia osób z jakąkolwiek niepełnosprawnością. Jednym z pierwszych urządzeń wspomagających niesłyszących był aparat słuchowy opracowywany przez Millera Reese'a Hutchisona od około 1895 roku [29]. Rozwiązaniem, które zapowiadało późniejszy rozwój elektronicznych systemów wspomagających był system POSSUM (ang. *Patient Operated Selector Mechanism*) opracowany początkiem lat 60. XX wieku. Umożliwiał on osobom z poważną niepełnosprawnością ruchu sterowanie maszyną do pisania [28]. Nie można jednak mówić o cyfrowych systemach wspomagających przed pojawieniem się elektronicznych komputerów. W latach 60. i 70. XX wieku zaczęły pojawiać się bardziej zaawansowane systemy wspomagające oparte o cyfrowe jednostki obliczeniowe. Przykładem jednego z nich może zostać Tufts Interactive Communicator, który umożliwiał niemym z ograniczeniami ruchowymi komunikację z otoczeniem poprzez wybieranie znaków za pomocą mechanizmu skanowania, a następnie prezentację pełnego komunikatu na wyświetlaczu lub w formie wydruku [53]. Późniejszy przykład podobnego systemu był wykorzystywany przez wybitnego fizyka Stephena Hawkinga, który pomimo swojej postępującej niepełnosprawności ruchowej, mógł sprawnie komunikować się z otoczeniem poprzez niewielkie ruchy policzka [17].

Równocześnie rozwijały się technologie wspomagające osoby z niepełnosprawnością wzroku, w których dane przekształcano na formę możliwą do odbioru przez inne zmysły. Kamieniem milowym okazało się urządzenie Optacon (ang. *Optical to Tactile Converter*) opatentowane w 1966 roku przez Johna G. Linvilla. Umożliwiała ono przekształcenie obrazu na vibracje matrycy, które mogły być odczytane przez użytkownika za pomocą dotyku [23]. W kolejnych dekadach rozwijano m.in. systemy rozpoznawania znaków, syntezy mowy, czytniki ekranów i urządzenia mobilne, stopniowo zwiększając dostęp osób niewidomych do informacji cyfrowych.

2.2.2 Standardy i regulacje prawne dotyczące dostępności

Rozwój technologiczny nie był jedynym czynnikiem, który przyczynił się do upowszechnienia systemów wspomagających. Istotną rolę odegrały również regulacje prawne

i standardy, które nakładały obowiązek zapewnienia dostępności systemów informatycznych dla osób z niepełnosprawnością. Bardzo istotnym krokiem było opublikowanie w 1999 roku przez World Wide Web Consortium (W3C) pierwszej wersji wytycznych WCAG (ang. *Web Content Accessibility Guidelines*) określających, w jaki sposób powinny być projektowane treści internetowe, by były one dostępne dla osób z różnymi rodzajami niepełnosprawności [57]. Kolejne wersje WCAG uporządkowały wytyczne wokół czterech głównych zasad: postrzegalności, funkcjonalności, zrozumiałości i solidności (kompatybilności). Najnowsza wersja standardu WCAG 2.2 została opublikowana 5 października 2023 [58]. Obecnie trwają prace nad kolejną wersją WCAG 3.0.

Znaczenie dostępności zostało następnie zwiększone dzięki wdrożeniu kroków prawnych. Artykuł 9 Konwencji ONZ o prawach osób z niepełnosprawnościami nakłada na państwa członkowskie obowiązek przystosowania dostępu do informacji, komunikacji, infrastruktury i technologii do potrzeb osób z niepełnosprawnością [52]. Unia Europejska wprowadziła w 2016 roku dyrektywę 2016/2102, która określa wymagania dotyczące dostępności stron internetowych i aplikacji mobilnych podmiotów sektora publicznego [10]. W Polsce wymagania wynikające z dyrektywy wdrożono ustawą z dnia 4 kwietnia 2019 roku o dostępności cyfrowej stron internetowych i aplikacji mobilnych podmiotów publicznych [42]. Od 28 czerwca 2025 roku w Polsce obowiązuje również Europejski Akt o Dostępności, który nakłada wymagania na wybrane usługi i produkty, takie jak urządzenia mobilne, komputery, czytniki książek, handel elektroniczny, bankowość i transport [11, 41]. Szczegółowe wymagania techniczne dla produktów i usług ICT określa norma EN 301 549, która jest stosowana w całej Unii Europejskiej [9].

Dla osób niewidomych te regulacje mają szczególne znaczenie w kontekście współpracy czytników ekranowych z interfejsami, możliwości obsługi bez użycia wzroku i jednoznacznego przekazu informacji. Rozwój prawa sprawił więc, że dostępność cyfrowa nie pozostała w fazie teoretycznej, ale stała się wymogiem, a przez to istotnym czynnikiem technologii wspomagających.

2.2.3 Współczesne systemy wspomagające osoby niewidome

Wraz z rozwojem technologii informatycznej i postępującym zwiększaniem dostępności cyfrowej zaczęły powstawać kompletne systemy wspomagające osoby niewidome, których rozmiary i możliwości zazwyczaj nie krępowały codziennego funkcjonowania. Dziś wiele z nich wykorzystuje uczenie maszynowe i kamery do analizy przestrzeni wokół użytkownika. Przykład ich użycia zaproponowali Potdar, Pai i Akolkar, którzy oparli swoją pracę na konwolucyjnych sieciach neuronowych (CNN) i kamerze. Zaznaczyli, że jednym z ich celów było, aby osoba niewidoma nie musiała wchodzić w bezpośredni kontakt z obiektem przed sobą, aby zminimalizować ryzyko uszczerbku na zdrowiu lub utraty życia. Sieć zastosowana przez autorów zawierała warstwy odpowiadające za wyodrębnianie

cech, lokalizację i klasyfikację obiektów. System umożliwia rozpoznawanie 200 klas obiektów obejmujących ludzi, pojazdy, zwierzęta i elementy wyposażenia pomieszczeń. Sens działania systemu jest prosty: użytkownik naciska przycisk, kamera rejestruje obraz, a następnie model CNN rozpoznaje obiekty i przekazuje użytkownikowi informacje o nich. Niestety, założenie o naciśnięciu przycisku wyklucza możliwość ciągłego rozpoznawania obiektów. Autorzy zaznaczają, że ich wyniki potwierdzają użyteczność systemu, ale jednocześnie mówią o utracie jakości przy małych obiektach. Sugerują, że zwiększenie liczby warstw konwolucyjnych może poprawić skuteczność, ale jednocześnie spowolni działanie systemu. Te wnioski pokazują, że należy zachować kompromis pomiędzy skutecznością a prędkością działania systemu [31].

Rozszerzeniem idei wykorzystania sieci neuronowych do wsparcia osób niewidomych jest system MagicEye opracowany przez zespół pod przewodnictwem Sethuramana. Tym razem twórcy oparli się na idei stworzenia urządzenia ubieralnego w formie inteligentnych okularów z kamerą. Podobnie jak w przypadku poprzedniego systemu, po wejściu w interakcję z urządzeniem obraz z kamery jest analizowany przez model sieci neuronowej, a następnie wyniki detekcji są przekazywane użytkownikowi w formie dźwiękowej. Model oparto na architekturze YOLOv5, wybranej ze względu na szybkość działania, skuteczność i niewielki rozmiar sieci. Trening przeprowadzono na wybranym podzbiorze Google Open Images liczącym ponad 13 200 obrazów, obejmującym 35 klas obiektów. Dodatkowo, w systemie zaimplementowano mechanizmy rozpoznawania twarzy, nominałów banknotów, odbiornik GPS i czujnik zbliżeniowy [44].

Kolejnym krokiem w rozwoju technologii asystujących jest już nie tylko rozpoznanie obiektów, ale przekazanie użytkownikowi informacji dotyczących kontekstu całej sceny. Takie rozwiązanie znajdujemy w pracy Baiga et al., którzy zaproponowali system oparty na Raspberry Pi 4 i model sztucznej inteligencji. Dzięki temu, nie byli ograniczeni do skończonego zbioru klas obiektów, ale dzięki użyciu dużego modelu vision-language mogli generować opisy całej sceny i przekazywać je użytkownikowi w formie audio. Dodatkowo, w systemie zaimplementowano rozpoznawanie twarzy i detekcję odległości od przeszkód. Zgodnie z założeniami, system działał w czasie rzeczywistym, ale radził sobie znacząco gorzej w złych warunkach oświetleniowych [1]. Ta praca rozwiązuje problem poruszony w podrozdziale 2.1, mówiący o tym, że sama informacja o obiekcie może być niewystarczająca. Tutaj twórcy skupili się na przekazaniu jak najpełniejszej informacji o przestrzeni, w której znajduje się użytkownik. Warto jednak zauważyć, że skorzystanie z dużego modelu wymaga ciągłego połączenia z Internetem, a lokalne działanie systemu jest ograniczone do rozpoznawania twarzy i detekcji pobliskich przeszkód.

Poprzednio omawiane systemy skupiały się na rozpoznawaniu obiektów w określonej scenie, natomiast głównym celem systemu zaproponowanego przez Wang et al. jest pomoc użytkownikowi w omijaniu przeszkód w przestrzeni. Ponieważ rozwiązanie ma działać jako ciągła pomoc nawigacyjna, nie powinno być aktywowane przyciskami, a zamiast tego stale

analizować obraz i – w razie potrzeby – informować użytkownika o potencjalnym zagrożeniu. Autorzy przedstawili metodę detekcji YOLO-OD do której wykorzystali model o architekturze inspirowanej siecią YOLOv8 rozszerzoną o dodatkowe bloki odpowiedzialne za zwiększenie skuteczności detekcji w różnych warunkach. Sam model na publicznie dostępnym zbiorze danych osiągnął wartość średniej precyzji na poziomie 30,02%, co zdaniem autorów wskazuje, że zaproponowane rozwiązanie może być wartościowe w kontekście wspomagania osób niewidomych [56].

Odrębnym zagadnieniem, na jakie należy zwrócić uwagę przy tworzeniu systemów wspomagających, jest sposób przekazywania informacji. Wcześniej omawiane rozwiązania bazowały na prostym przekazie dźwiękowym, opartym o buzzer lub syntezytor mowy. Dzierzgowska et al. zaproponowali metodę audio-graficzną, łączącą interaktywną prezentację wykresów funkcji matematycznych z opisami dźwiękowymi. Użytkownik, aby uzyskać kolejne informacje o wybranej części wykresu, musiał wykonać pojedyncze, podwójne lub potrójne stuknięcie na elemencie. Badanie przeprowadzone na grupie 54 osób wskazało, że zaproponowana metoda umożliwiła zmniejszenie obciążenia, frustracji, a w przypadku części zadań pozwoliła na skrócenie czasu ich wykonania. Wyniki wskazują, że skuteczność systemu wspomagającego nie zależy jedynie od stopnia możliwości rozpoznawania obiektów, ale również od poprawnego i odpowiedniego zaprojektowania przekazu informacji zwrotnej [8].

Współczesne technologie wspomagające uzupełniają tradycyjne metody wsparcia osób niewidomych, oferując coraz bardziej złożone systemy i algorytmy. Wspólnym mianownikiem większości z nich jest wykorzystanie małych kamer do pozyskiwania obrazu, który następnie jest analizowany przez modele uczenia maszynowego, a wyniki są prezentowane użytkownikowi. Większość opisywanych systemów koncentruje się jednak na wsparciu w codziennym funkcjonowaniu – poruszaniu się w przestrzeni, wykrywaniu przeszkód czy rozpoznawaniu twarzy i banknotów. Niewiele systemów skupia się jedynie na obiektach aktualnie eksplorowanych dotykowo przez użytkownika. W tym kontekście stworzenie systemu dedykowanego nauce przestrzennej wydaje się być istotnym uzupełnieniem istniejących rozwiązań.

2.3 Problem rozpoznawania obiektów 3D w interakcji dotykowej

Problemem, jaki należy koniecznie poruszyć, jest rozpoznanie obiektów 3D w interakcji dotykowej, ponieważ zachodzi w tym miejscu istotna różnica od systemów opisywanych we wcześniejszym podrozdziale (2.2.3). W odróżnieniu od nich, głównym celem projektowanego systemu nie jest rozpoznanie obiektów przed użytkownikiem, a tych, z którymi

wchodzi w interakcję. Mamy więc do czynienia z sytuacją, w której kamera nie jest zwrócona na to, co znajduje się przed użytkownikiem, ale na niego samego, a w szczególności na jego dłoń. W tym kontekście detekcja staje się bardziej wymagająca ze względu na to, że spora część obiektu może być przez nie zasłonięta, a sam kształt może być trudny do rozpoznania nawet dla osoby widzącej. W tym miejscu warto przytoczyć kilka prac, które poruszają podobne zagadnienia.

Pierwszą z nich jest badanie przeprowadzone przez Shan et al., którzy sprawdzali zdolność modelu do rozpoznawania dłoni w kontakcie z obiektami. W tym celu przygotowali zbiór danych ponad 100 tys. obrazów, na których oznaczono ponad 180 tys. dłoni i ponad 110 tys. obiektów. Celem zaproponowanego modelu nie było jednak określenie klasy dotykane go przedmiotu, a jak najlepsze wskazanie pozycji dłoni i jej orientacji. Dzięki temu, możliwe było wyznaczenie nie tyle położenia przedmiotu, a formy relacji, w jaką wchodził z dłonią. Detekcja została oparta o model Faster R-CNN, który dla pełnej predykcji wymagającej poprawnego wykrycia dłoni i obiektu uzyskał 38,5 AP50 [45]. Pokazuje to, że w przypadku wykrywania trzymany ch obiektów należy zwrócić uwagę na ich otoczenie, gdyż może dojść do sytuacji, gdy model poprawnie rozpozna obiekt, ale nie będzie on trzymany przez użytkownika, co może prowadzić do błędnych wyników.

Kolejnym aspektem na jaki należy zwrócić uwagę jest możliwość zasłonięcia części obiektu przez dłoń użytkownika. W odróżnieniu od poprzedniej pracy, Zhang et al. skupili się na rekonstrukcji trójwymiarowej przedmiotów trzymany ch w dłoniach. Podczas uczenia wykorzystywali dane z wielu widoków, aby ograniczyć wpływ zasłaniania obiektu przez dłoń. Autorzy wskazali dwa źródła niepełności informacji: zasłonięcie części przedmiotu przez dłoń lub niewidoczność części obiektu wynikająca z obserwowania go tylko pod jednym kątem. Wskazuje to, że w pojedynczej klatce nawet dobrze widoczny przedmiot może nie ujawniać wszystkich swoich cech [61]. Przez to istotne jest, aby zbiór danych treningowy ch zawierał bardzo zróżnicowane kombinacje ułożenia dłoni i obiektu. Warto zwrócić uwagę na to, że możliwa jest również analiza obiektu z kilku ujęć, co może pozwolić na odtworzenie jego widocznej części w przestrzeni trójwymiarowej. W literaturze znajdujemy również propozycje analizy sekwencji obrazów, która pozwoli na ograniczenie problemu zasłaniania obiektu, a możliwe, że również wykluczy niektóre przypadki błędnej predykcji. Jedną z prac poruszających ten problem jest badanie zespołu pod przewodnictwem Tekina. Zaproponowali oni system analizujący sekwencje obrazów RGB rejestrowany ch przez użytkownika z perspektywy pierwszej osoby. Model jednocześnie określa położenie dłoni i obiektu, rozpoznaje jego klasę i wykonywaną czynność. Kolejne klatki są następnie analizowane przez moduł rekurencyjny, badający zmiany położenia wykrywanyc h dłoni i obiektów. Autorzy zaznaczają, że takie podejście pozwoliło im na poprawę skuteczności rozpoznawania czynności z 85,56% do 96,99% w porównaniu z analizą pojedynczy ch klatek. System działał również w czasie rzeczywistym z prędkością około 25 klatek na

sekundę [48]. Wskazuje to, że wykorzystanie informacji z kolejnych klatek może stanowić sposób ograniczenia wpływu chwilowych zasłonień obiektów.

Na problem rozpoznawania obiektów można również spojrzeć z perspektywy robotyki. Możliwa jest eksploracja obiektu bez użycia kamer, a jedynie poprzez dotyk. Mimo że w tym przypadku nie mamy bezpośredniego odniesienia do osób niewidomych, to warto zwrócić uwagę na inne sposoby akwizycji danych. Ramię robota stworzonego przez zespół Xu et al. wyposażono w czujnik dotyku, a następnie umożliwiono mu eksplorację 10 przedmiotów ze zbioru YCB. Każde dotknięcie tworzyło wpis w lokalnej chmurze punktów, która następnie była wykorzystywana do predykcji lub określenia kolejnego ruchu pozwalającego na lepsze rozróżnienie przedmiotów. Autorzy wskazali, że już kilka charakterystycznych punktów pozwalało na odróżnienie obiektów [59]. Sugeruje to, że kamery nie są jedynym sposobem na pozyskanie informacji, a ciekawym kierunkiem badań może być analiza chmury punktów powstałej w wyniku dotyku lub skanowania przestrzeni. Odpowiednikiem wspomnianego robota może być rękawica sensoryczna, która będzie zbierała dane nie tylko o samym obiekcie, ale również o rękach użytkownika. Zhang et al. opracowali właśnie takie rozwiązanie, które na podstawie zbieranych danych mogło rozpoznać 16 klas obiektów z dokładnością ponad 95% [62]. Ukazuje to, że do rozpoznania obiektów nie musimy wykorzystywać wyłącznie obrazów, a możliwe jest połączenie różnych źródeł danych, takich jak obraz, głębia czy informacje dotykowe. Jednakże rozwiązania wykorzystujące czujniki dotykowe wymagają dodatkowego wyposażenia, które przez utworzenie dodatkowej warstwy na dłoniach użytkownika może utrudniać mu eksplorację obiektu. W dalszej analizie skoncentrowano się na sposobach pozyskiwania danych możliwych za pomocą urządzeń, które nie wchodzi w bezpośredni kontakt z użytkownikiem.

2.4 Metody detekcji obiektów w obrazach

Mimo że dzisiejsza technika detekcji obiektów w obrazach opiera się głównie na sieciach neuronowych, istnieją różne podejścia do tego problemu. Należy wspomnieć o klasycznych metodach wizji komputerowej, wykorzystujących cechy samego obrazu, takie jak krawędzie, kolory czy histogramy [65]. W niniejszej sekcji przedstawiono przegląd najważniejszych metod detekcji obiektów.

2.4.1 Sformułowanie problemu detekcji obiektów

Detekcja jest zadaniem trudniejszym od klasyfikacji obrazu, ponieważ oprócz określenia, jakie obiekty znajdują się na obrazie, musi również wskazać ich położenie. Sama klasyfikacja opiera się na przypisaniu jednej lub kilku etykiet do całego obrazu, co w wielu przypadkach może być niewystarczające. Lokalizacja rozszerza klasyfikację o wyznaczenie

ramki ograniczającej (ang. *bounding box*) określającej położenie obiektu, najczęściej zakładając pojedynczą instancję. Zadaniem detekcji jest więc odnalezienie wszystkich istotnych obiektów na obrazie, a następnie przypisanie odpowiadających im klas i określenie ich położenia [37]. Wynik działania detektora może być przedstawiony jako zbiór predykcji:

$$D = \{(c_i, b_i, p_i)\}_{i=1}^N, \quad (2.1)$$

gdzie N jest liczbą wykrytych obiektów, c_i to przewidywana klasa i -tego obiektu, b_i opisuje jego ramkę, a p_i jest wartością pewności predykcji.

Ramka ograniczająca najczęściej jest prostokątem opisanym współrzędnymi dwóch przeciwległych narożników lub częściej współrzędnymi środka, szerokości i wysokości:

$$b_i = (x_i, y_i, w_i, h_i), \quad (2.2)$$

gdzie x_i i y_i określają położenie środka ramki, a w_i i h_i jej szerokość oraz wysokość. Zależnie od implementacji każda z tych wartości może być wyrażona w pikselach lub znormalizowana względem rozmiaru obrazu. Drugie podejście uniezależnia reprezentację ramki od rozdzielczości obrazu wejściowego i jest powszechnie stosowane przy adnotacjach i predykcji.

Wartość pewności predykcji, w zależności od implementacji, określa stopień przekonania modelu, że dany obszar zawiera obiekt przewidywanej klasy. Jeżeli ta wartość nie przekracza przyjętego progu, wynik wykonanej predykcji może zostać odrzucony. Jest to więc istotny parametr, jednak nie przesądza on poprawności predykcji. Przy ocenie należy również spojrzeć na zgodność klasy przewidywanej ze stanem faktycznym oraz na stopień pokrywania się ich ramek ograniczających (ang. *bounding box*). Zagadnienia te są szczegółowo opisane w podrozdziale 2.5, dotyczącym metryk oceny detekcji.

Zadaniem powiązanim z detekcją obiektów jest segmentacja instancji, której celem oprócz wyznaczenia ramki i klasy każdego obiektu jest określenie ich masek pikselowych. Umożliwia to dokładne określenie kształtu obiektów, ale wymaga przygotowania dokładniejszych danych uczących [16]. W przypadku projektowanego systemu zadaniem modelu jest wykrycie trzymanej bryły geometrycznej, określenie jej klasy, położenia oraz wartości pewności predykcji. Istotna przy tym jest nie tylko dokładność predykcji, ale szczególnie szybkość działania, gdyż wynik ma być przekazywany użytkownikowi w momencie jego bezpośredniej interakcji z obiektem. W tym kontekście segmentacja instancji może nałożyć za duże wymagania obliczeniowe, a jej zastosowanie może nie wiązać się z istotną poprawą jakości.

2.4.2 Klasyczne metody wizji komputerowej

Zanim w detekcji obiektów upowszechniły się głębokie sieci neuronowe, wykorzystywano przede wszystkim ręcznie projektowane metody przetwarzania obrazów. Między innymi wykorzystywano progowanie, segmentację na podstawie koloru, detekcję krawędzi i analizę konturów. Pozwalało to na wyodrębnienie obszarów różniących się od tła i opisanie ich cech, takich jak rozmiar, proporcje czy kształt. W kontrolowanych warunkach te metody mogły być skuteczne, jednak obiekty musiały wyraźnie odróżniać się od tła.

Późniejsze, bardziej rozbudowane podejścia wykorzystywały deskryptory cech, jak histogramy zorientowanych gradientów (HOG) czy skalo-niezmienne przekształcenie cech (SIFT) [6, 26]. Deskryptory obliczano dla całego obrazu lub kolejnych jego fragmentów za pomocą przesuwanego okna, a następnie wyodrębnione cechy przekazywano do klasyfikatora, którym mogła być maszyna wektorów nośnych (SVM). Warto również zwrócić uwagę na kaskady klasyfikatorów wykorzystujące cechy podobne do cech Haara, pozwalające na szybkie wykrywanie obiektów, głównie wykorzystywane w metodzie Viola–Jonesa [54].

Największe zalety metod klasycznych ujawniają się w ich prostocie, stosunkowo niskim koszcie obliczeniowym oraz możliwości interpretacji kolejnych etapów działania. Niestety, wymagają one ręcznego wyboru cech i ciągłego dostosowywania parametrów, aby uzyskać jak najlepsze wyniki. Są również bardzo wrażliwe na zmieniające się warunki otoczenia – niewielkie zmiany w oświetleniu, tle czy częściowe zasłonięcie obiektu mogą skutkować krytycznym spadkiem skuteczności. Te ograniczenia uzasadniają wykorzystanie głębokich sieci neuronowych, które w wielu przypadkach radzą sobie lepiej i mogą same, automatycznie uczyć się cech z danych treningowych.

2.4.3 Detektory dwuetapowe

Z czasem rozwój metod uczenia głębokiego doprowadził do coraz częstszego zastępowania ręcznie projektowanych deskryptorów cech przez ich reprezentacje automatycznie określone przez CNN. Jedną z rodzin takich rozwiązań są detektory dwuetapowe. Ich działanie podzielone jest na dwa główne etapy: wyznaczenie obszarów zainteresowań, a następnie ich klasyfikacja i dopasowanie położenia ramek ograniczających.

Jednym z pierwszych detektorów dwuetapowych był model R-CNN wykorzystujący algorytm do wyznaczenia propozycji regionów, które następnie kolejno były przetwarzane przez sieć konwolucyjną. Niestety, prowadziło to do wielokrotnego sprawdzania tych samych fragmentów obrazu przez sieć, co skutkowało dużym kosztem obliczeniowym. Kolejna wersja, Fast R-CNN, wprowadziła jednokrotne przetwarzanie całego obrazu przez sieć, a cechy były pobierane ze wspólnej mapy cech. Ograniczyło to liczbę wykonywanych obliczeń, ale wciąż generowano propozycje regionów za pomocą algorytmu. Dalszym rozwinięciem było wprowadzenie wersji Faster R-CNN, w którym zastąpiono algorytm modułem sieci propozycji regionów RPN (ang. *Region Proposal Network*), która analizuje

mapę cech i wskazuje obszary zawierające potencjalne obiekty. Oba moduły korzystały ze wspólnych cech obrazu, co pozwoliło znacznie ograniczyć koszt obliczeniowy [39].

Detektory dwuetapowe mogą osiągać wysoką skuteczność i dokładność lokalizacji dzięki rozdzieleniu od siebie etapu generowania propozycji regionów od ich klasyfikacji i dopasowania ramek ograniczających. Niestety, ten podział wpływa na złożoność obliczeniową całego modelu, a w konsekwencji na jego czas działania. W wielu przypadkach może stanowić to istotną przeszkodę, szczególnie jeżeli wynik ma być otrzymywany w czasie rzeczywistym.

2.4.4 Detektory jednoetapowe

Ostatnia dekada przyniosła znaczny rozwój detektorów jednoetapowych, których rdzeń działania opiera się na przewidywaniu klas obiektów i ich ramek ograniczających w pojedynczym kroku. Za pierwszy nowoczesny przykład takiego podejścia uznaje się OverFeat z 2013 roku [43], jednakże dopiero modele wprowadzone po roku 2015 zyskały wysoką popularność i rozpoznawalność. Mimo że historycznie detektory jednoetapowe oferowały mniejszą dokładność od detektorów dwuetapowych, ich główną zaletą była szybkość działania, co umożliwiło ich wykorzystanie w systemach czasu rzeczywistego (ang. *Real-Time*, RT) [21, 65].

Jednym z pierwszych znaczących zaproponowanych modeli jest SSD (ang. *Single Shot MultiBox Detector*) zaprezentowany na konferencji ECCV w 2016 roku przez Wei Liu. Jego działanie opiera się na przewidywaniu klas obiektów i położenia ramek ograniczających na kilku mapach o różnej rozdzielczości. Większe mapy umożliwiają wykrywanie mniejszych obiektów, a głębsze o mniejszej rozdzielczości służą rozpoznawaniu obiektów większych. Model wykorzystuje domyślne ramki o różnych proporcjach i skalach, których współrzędne następnie koryguje. Taki mechanizm umożliwia wykrycie obiektów o różnych rozmiarach bez osobnego etapu generowania propozycji regionów [25]. Głównym ograniczeniem modelu SSD stała się zależność skuteczności detekcji od odpowiedniego doboru ramek, a także spadek dokładności przy wykrywaniu małych obiektów.

W podobnym czasie został zaprezentowany również model YOLO (ang. *You Only Look Once*), w którym zadanie detekcji przedstawiono jako pojedynczy problem regresji. Takie podejście doprowadziło do modelu, który przewiduje ramki ograniczające oraz prawdopodobieństwa klas w oparciu bezpośrednio o obraz wejściowy [37]. Uproszczenie i ujednolicenie procesu umożliwiło uzyskanie krótkiego czasu działania, co z kolei sprawiło, że modele YOLO znalazły zastosowanie w systemach czasu rzeczywistego. Kolejne lata przyniosły rozwój rodziny modeli YOLO, w której znalazły się warianty o różnej liczbie parametrów i złożoności obliczeniowej, umożliwiając wybranie modelu dopasowanego do wymagań i ograniczeń systemu. Szczegółowy opis rodziny modeli YOLO znajduje się w podrozdziale 2.6.

2.4.5 Detektory oparte na architekturze transformera

Podjęciem alternatywnym do detektorów opartych na sieciach konwolucyjnych są modele oparte na architekturze transformera. Podstawowym elementem tego podejścia jest mechanizm samo-uwagi (ang. *self-attention*), który pozwala na analizę zależności pomiędzy różnymi fragmentami danych wejściowych. Podczas działania modelu Vision Transformer obraz wejściowy jest dzielony na mniejsze fragmenty, które następnie są reprezentowane jako sekwencja wektorów i przetwarzane przez warstwy transformera. Takie podejście pozwala na znajdowanie relacji pomiędzy odległymi od siebie elementami obrazu. Użycie transformerów wizyjnych wymaga również przygotowania dużych zbiorów danych treningowych tak, aby model mógł nauczyć się zależności globalnych [7].

Pierwszą propozycją zastosowania tego podejścia w detekcji obiektów jest model DETR (ang. *DEtection TRansformer*) zaprezentowany przez zespół będący częścią Facebook AI Research pod kierownictwem Cariona w 2020 roku. Jego podstawowa wersja opiera się na wyznaczeniu podstawowych cech przez CNN, a następnie przekazaniu ich do transformera zrealizowanego w architekturze enkoder-dekoder. W dekodzie wykorzystywany jest zbiór zapytań obiektowych (ang. *object queries*), na których podstawie model dokonuje równoległej predykcji klas obiektów i ich ramek ograniczających. Wynik detekcji jest traktowany przez model jako zbiór predykcji, które w trakcie uczenia są dopasowywane do obiektów referencyjnych poprzez dopasowanie dwudzielne (ang. *bi-partite matching*) przy pomocy algorytmu węgierskiego. Pozwala to na uniknięcie użycia ramek domyślnych i algorytmu NMS (ang. *Non-Maximum Suppression*). W stosunku do klasycznych detektorów, DETR upraszcza potok detekcji, co pozwala na jego uczenie jako pojedynczego rozwiązania end-to-end. Niestety zalety płynące z użycia tego podejścia w detekcji wiążą się z wysokim kosztem treningu i trudniejszą optymalizacją modelu, szczególnie w przypadku wykrywania małych obiektów [3].

2.4.6 Porównanie metod detekcji

Przytoczone metody detekcji obiektów różnią się od siebie przede wszystkim sposobem wyznaczania położenia obiektów na obrazie, złożonością obliczeniową oraz ich stosunkiem pomiędzy dokładnością a szybkością działania. Zestawienie ich najważniejszych właściwości, ważnych z punktu widzenia systemu, przedstawiono w tabeli 2.1.

Zestawienie nie wskazuje jednego najlepszego rozwiązania, ale przedstawia zalety i wady poszczególnych metod. Wybór podejścia zależy od nałożonych wymagań oraz ograniczeń sprzętowych i środowiskowych. W projektowanym systemie szczególnie ważny jest krótki czas reakcji, odporność na zmieniające się warunki środowiskowe oraz zachowanie odpowiedniej skuteczności. Tych założeń – w większości – nie spełniają, mimo niewielkich wymagań, rozwiązania klasyczne będąc silnie zależnymi od ręcznie dobieranych cech

Tabela 2.1: Porównanie wybranych metod detekcji obiektów

Metoda	Charakterystyka	Atuty	Ograniczenia
Metody klasyczne	Ręcznie projektowane cechy i osobne etapy przetwarzania	Niewielkie wymagania obliczeniowe, interpretowalność i możliwość działania w kontrolowanych warunkach	Konieczność ręcznego określania cech, wrażliwość na zmianę oświetlenia, tła, orientacji i częściowe zasłonięcie obiektu
Faster R-CNN	Dwuetaapowe generowanie propozycji regionów, ich klasyfikacja i dopasowanie ramek	Wysoka skuteczność i dokładne określenie położenia obiektów	Większa złożoność obliczeniowa i opóźnienie wynikające z przetwarzania dwuetaowego
SSD	Jednoetaapowe predykcje na mapach cech o różnych rozdzielczościach	Krótki czas odpowiedzi i możliwość wykrywania obiektów o różnych rozmiarach	Zależność od konfiguracji ramek domyślnych i gorsze wyniki dla małych obiektów
YOLO	Jednoetaapowa predykcja klas i ramek ograniczających	Szybkie działanie, możliwość pracy w czasie rzeczywistym i dostępność wariantów modelu	Konieczny kompromis pomiędzy szybkością a skutecznością, szczególnie przy małych lub częściowo zasłoniętych obiektach
DETR	Bezpośrednia predykcja zbioru obiektów z wykorzystaniem transformera i mechanizmu uwagi	Modelowanie globalnych zależności w scenie i brak konieczności używania ramek domyślnych ani algorytmu NMS	Długi trening i trudniejsza optymalizacja

i warunków. Detektory dwuetaapowe mogą natomiast osiągać wysoką skuteczność, ale oddzielne generowanie i przetwarzanie skutkuje zwiększeniem ich złożoności obliczeniową i czasu uzyskania odpowiedzi. Z tych względów uwagę należy kierować na detektory jednoetaapowe, które oferują kompromis między czasem i skutecznością detekcji. Predykcje o wartości pewności niższej od ustalonego progu mogą zostać odrzucone, ograniczając ryzyko przekazania użytkownikowi wyników, których model nie jest dostatecznie pewny [32].

Spośród detektorów jednoetaapowych wymagania projektowanego systemu najlepiej spełnia rodzina modeli YOLO. W pojedynczym kroku detekcji potrafią one bezpośrednio przewidzieć klasę obiektu i jego położenie, co znacznie skraca czas potrzebny na uzyskanie wyniku. Rodzina zapewnia duży wybór wariantów modeli, dopasowanych do konkretnych

zadań i zdolności obliczeniowej urządzeń. Pierwsza wersja YOLO osiągnęła prędkość predykcji około 45 FPS (ang. *frames per second*), a jej uproszczone warianty nawet 155 FPS, co potwierdziło, że może zostać wykorzystana w rozwiązaniach RT [37].

Wykorzystanie detektora opartego na architekturze transformera nie jest w tym wypadku uzasadnione. Analizowana scena ma zawierać niewielką liczbę obiektów, a zadaniem modelu jest przede wszystkim określenie ich lokalizacji i klasy. Jawne modelowanie globalnych zależności nie stanowi kluczowego wymagania, a korzyści płynące z tej metody nie równoważą dodatkowego kosztu, który należy ponieść przy uczeniu i optymalizacji modelu [3].

2.5 Metryki oceny detekcji obiektów

Ocena modeli detekcji obiektów wymaga zastosowania odpowiednich metryk, które pozwalają na porównanie uzyskanych wyników. Metryki te powinny uwzględniać zarówno poprawność klasyfikacji, jak i dokładność lokalizacji obiektu na obrazie. W przeciwieństwie do klasyfikacji obrazów, przy detekcji nie wystarczy, aby model poprawnie rozpoznał klasę obiektu. Równie istotne jest, aby poprawnie wyznaczył ramkę ograniczającą wokół wykrytego obiektu. Jedną z najczęściej stosowanych metryk jest IoU (ang. *Intersection over Union*), określająca stopień pokrycia między ramką obiektu przewidywaną przez model a ramką rzeczywistą [30]. Wartość IoU obliczana jest jako stosunek pola części wspólnej obu ramek do pola ich części łącznej:

$$IoU = \frac{|B_p \cap B_{gt}|}{|B_p \cup B_{gt}|}, \quad (2.3)$$

gdzie B_p jest ramką przewidywaną przez model, a B_{gt} jest ramką referencyjną (ang. *ground truth*). Im większa wartość IoU, tym lepsze dopasowanie ramki przewidywanej do ramki rzeczywistej. W praktyce najczęściej stosuje się progi, np. $IoU \geq 0,5$, aby zdecydować, czy dana detekcja może być uznana za poprawną. Próg ten został wykorzystany w wielu klasycznych procedurach oceny detekcji, takich jak PASCAL VOC [12], i jest nadal powszechnie stosowany w wielu benchmarkach. Na podstawie wartości IoU oraz zgodności klasy obiektu możliwe jest przypisanie wyników detekcji do jednej z kategorii: prawdziwych pozytywów (TP, ang. *true positive*), fałszywych pozytywów (FP, ang. *false positive*) i fałszywych negatywów (FN, ang. *false negative*). Detekcja jest traktowana jako TP, jeżeli model poprawnie rozpoznał klasę obiektu, a przewidywana ramka spełnia zadany próg IoU względem ramki referencyjnej. FP to detekcja błędna, w której model przewidział klasę obiektu, ale nie spełnił progu IoU lub przewidziana klasa była błędna. FN natomiast to sytuacja, w której model nie wykrył obiektu, który rzeczywiście był na obrazie.

Jedną z podstawowych metryk oceny jest precyzja (ang. *precision*), określająca udział poprawnych detekcji modelu wśród wszystkich detekcji, jakie zwrócił:

$$Precision = \frac{TP}{TP + FP}. \quad (2.4)$$

Wysoka precyzja oznacza, że model generuje niewiele fałszywych pozytywów. Drugą, powiązaną z nią metryką, jest czułość (ang. *recall*), określająca, jaki jest udział poprawnych detekcji wśród wszystkich rzeczywistych obiektów:

$$Recall = \frac{TP}{TP + FN}. \quad (2.5)$$

Obie metryki pozostają ze sobą w relacji kompromisu, ponieważ zwiększenie progu pewności detekcji może ograniczyć liczbę fałszywych pozytywów, ale jednocześnie prowadzić do pominięcia części rzeczywistych obiektów. Z kolei obniżenie tego progu może zwiększyć liczbę detekcji, ale równocześnie powodować wzrost liczby fałszywych detekcji. Z tego względu, przy analizie detektorów często łączy się je w jedną metrykę – krzywą PR (precyzja-czułość, ang. *precision-recall curve*), przedstawiającą zależność pomiędzy nimi dla różnych wartości progu decyzyjnego. Na jej podstawie wyznacza się metrykę AP (ang. *Average Precision*), odpowiadającą polu pod krzywą PR.

W przypadku detekcji wieloklasowej używa się średniej wartości AP obliczonej dla wszystkich klas, określanej jako *mAP* (ang. *mean Average Precision*):

$$mAP = \frac{1}{N} \sum_{i=1}^N AP_i, \quad (2.6)$$

gdzie N jest liczbą klas, a AP_i jest wartością AP dla i -tej klasy. W praktyce najczęściej wykorzystuje się dwie wersje tej metryki: *mAP*50, gdzie detekcję uznaje się za poprawną przy progu $IoU \geq 0,5$, oraz *mAP*50-95, gdzie wynik jest uśredniany dla progów IoU z zakresu od 0,5 do 0,95 ze skokiem 0,05. Pierwsza z nich jest bardziej liberalna i pozwala określić ogólną zdolność modelu do wykrycia obiektu. Druga natomiast jest bardziej rygorystyczna, ponieważ wymaga dokładniejszego dopasowania ramek ograniczających, dzięki czemu lepiej pokazuje zarówno skuteczność klasyfikacji, jak i lokalizacji obiektu. Uśrednianie wyników dla wielu progów zostało szeroko zastosowane m.in. w metodyce oceny detekcji zbioru Microsoft COCO [22] i jest obecnie standardem w wielu benchmarkach detekcji obiektów.

Dodatkowo, w pracy wykorzystano metrykę *F1-score*, będącą średnią harmoniczną precyzji i czułości:

$$F_1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}. \quad (2.7)$$

Metryka ta umożliwia przedstawienie kompromisu pomiędzy liczbą fałszywych detekcji a liczbą pominiętych obiektów. Może być szczególnie przydatna, gdy sama precyzja lub czułość nie dają pełnego obrazu jakości modelu. W niniejszej pracy metrykę $F1$ -score wykorzystano jako metrykę pomocniczą względem podstawowych metryk $mAP50$ i $mAP50-95$, aby lepiej zrozumieć kompromis pomiędzy precyzją a czułością w kontekście detekcji obiektów dotykanych przez użytkownika.

Oprócz wartości liczbowych wykorzystano również macierze pomyłek (ang. *confusion matrix*), pozwalające przeanalizować, które klasy obiektów są najczęściej mylone przez model. Są one przedstawiane w formie tabel, w których jedna oś odpowiada rzeczywistym klasom, a druga klasom przewidywanym przez model. Dla dobrze działającego modelu większość wartości powinna znajdować się na przekątnej macierzy, co wskazuje na poprawną klasyfikację obiektów. Macierze pomyłek są szczególnie istotne przy rozpoznawaniu brył geometrycznych o podobnych kształtach, np. sześcianu i prostopadłościanu, ponieważ sama wartość mAP nie wskazuje bezpośrednio, pomiędzy którymi klasami model popełnia najwięcej błędów. Analiza macierzy pomyłek może również pomóc w identyfikacji problemów z danymi treningowymi, np. niedostatecznej liczby przykładów dla wybranych klas lub błędów w oznaczeniach.

2.6 Rodzina modeli YOLO

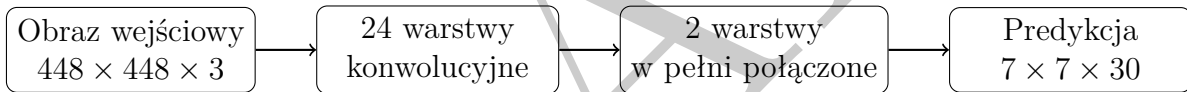
Pierwsza wersja YOLO zadebiutowała wraz z publikacją artykułu *You Only Look Once: Unified, Real-Time Object Detection* przez Josepha Redmona, Santosha Divvalę, Rossa Girshicka i Ali Farhadi w 2016 roku i od tego czasu rodzina modeli YOLO należy do najbardziej rozpoznawalnych i popularnych detektorów jednoetapowych. Autorzy określili problem detekcji obrazu jako regresję, w której pojedyncza sieć neuronowa przewiduje zarówno ramki obiektów, jak i odpowiadające im prawdopodobieństwa klas [37]. Kolejne lata przyniosły rozwój kolejnych modeli rodziny YOLO przez różne zespoły badawcze, więc nie istnieje pojedyncza gałąź rozwoju rodziny. Jedną z głównych gałęzi jest rozwijana przez firmę Ultralytics, która w 2020 roku zaprezentowała YOLOv5, a w następnych latach jej kolejne generacje. W chwili opracowywania pracy najnowszą wersją jest YOLOv6 [19].

2.6.1 Architektura modeli YOLO

Mimo że architektura każdej wersji modelu YOLO różni się od siebie, to idea pierwszej wersji pozostaje niezmienną. Założenie, że obraz jest analizowany w pojedynczym przebiegu sieci neuronowej, a wynik wyznaczany jest bez wcześniejszego wygenerowania propozycji regionów jest obecne w każdej kolejnej wersji. Sama architektura sieci mocno zmieniała się na przestrzeni kolejnych wersji, a porównanie kilku kolejnych modeli pozwala określić kierunek rozwoju całej rodziny.

Pierwszy model YOLO składał się z CNN złożonej z 24 warstw konwolucyjnych i dwóch warstw w pełni połączonych. Zadaniem warstw konwolucyjnych była ekstrakcja cech z obrazu wejściowego, a warstwy końcowe generowały wynik detekcji. W modelu wykorzystano m.in. warstwy 1×1 do redukcji liczby kanałów oraz warstwy 3×3 [37].

Dla obrazów ze zbioru PASCAL VOC wejście sieci miało rozmiar 448×448 pikseli, które następnie były przekształcane w logiczną siatkę o wymiarach $S \times S$. W zastosowanej konfiguracji sieci przyjęto wartość $S = 7$ dzielącą obraz na 49 komórek. Wykrycie obiektu następowało w oparciu o komórkę, w której znajdował się środek jego ramki ograniczającej. Każda z komórek mogła przewidzieć $B = 2$ ramki, wartości ich pewności i prawdopodobieństwa $C = 20$ klas obiektów występujących w zbiorze. Rozmiar tensora był definiowany jako $S \times S \times (B \cdot 5 + C)$, co w omawianym przypadku dawało tensor o wymiarach $7 \times 7 \times 30$. Każda ramka opisana była za pomocą pięciu wartości: współrzędnych środka ramki, jej szerokości i wysokości oraz wartości pewności predykcji. Jednoczesne przewidywanie ramek i klas obiektów w pojedynczym kroku wyeliminowało wymaganie stosowania oddzielnych etapów generowania propozycji regionów i ich klasyfikacji. Schemat przedstawiony na rysunku 2.1 pokazuje uproszczoną architekturę pierwszego modelu YOLO, w której, pomimo stosunkowo prostej budowy, udało się połączyć w jeden model zarówno klasyfikację, jak i lokalizację obiektów.



Rysunek 2.1: Uproszczony schemat architektury pierwszego modelu YOLO [37].

Kolejne wersje YOLO przyniosły przede wszystkim zmiany w architekturze przez stosowanie coraz wydajniejszych modułów sieci, korzystanie z kilku skal obrazu i modyfikację sposobu generowania ramek ograniczających. Z czasem architekturę opartą na pojedynczej siatce zastąpiono architekturami wykorzystującymi wieloskalowe mapy cech. Zmodernizowano również sposób w jaki łączone są informacje z różnych warstw sieci, a także budowę głowic. Dzięki temu, współczesne modele można przedstawić za pomocą trzech głównych elementów: *backbone*, *neck* i *head*. Najważniejsze wersje i zmiany w ich architekturze zestawiono w tabeli 2.2.

Współczesna architektura modelu YOLO26 zachowała konwolucyjny rdzeń rodziny, ale znacząco różni się od pierwszej wersji modelu. Sieć podzielona jest na trzy główne części, z których każda pełni inną funkcję. Pierwszą częścią jest *backbone*, który ma za zadanie ekstrakcję hierarchicznych reprezentacji cech z obrazu. Przechodzenie przez kolejne warstwy sieci wiąże się ze zmniejszeniem rozdzielczości przestrzennej map cech, a jednocześnie zwiększeniem zdolności do rozpoznawania coraz bardziej złożonych i abstrakcyjnych cech. Kolejna część to *neck* odpowiedzialny za połączenie informacji pochodzących z różnych poziomów sieci, co jest kluczowe przy wykrywaniu obiektów o różnych wymiarach: cechy pochodzące z głębszych warstw są bardziej abstrakcyjne, a płytsze warstwy dostarczają

Tabela 2.2: Wybrane etapy rozwoju architektury modeli YOLO. Opracowanie własne na podstawie [19, 37, 38, 51].

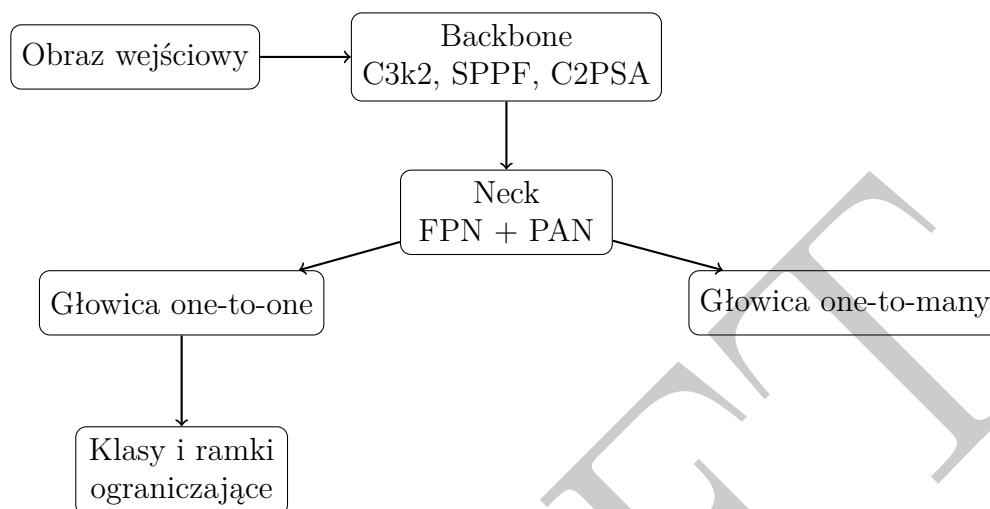
Generacja	Ekstrakcja cech	Sposób detekcji	Charakterystyczna zmiana
YOLO (v1)	Własna sieć CNN	Jedna siatka predykcji	Ujęcie detekcji jako pojedynczego problemu regresji
YOLOv3	Darknet-53	Detekcja na trzech skalach, ramki kotwiczące	Wprowadzenie wieloskalowej detekcji i architektury FPN
YOLOv5	CSP/C3	Wieloskalowa, ramki kotwiczące	Rozbudowanie fuzji cech o ścieżkę PAN i moduł SPPF
YOLOv8	C2f	Wieloskalowa, bez ramek kotwiczących	Przejsięcie na głowicę anchor-free i wykorzystanie DFL
YOLO11	C3k2, C2PSA	Wieloskalowa, bez ramek kotwiczących	Wprowadzenie kolejnych mechanizmów uwagi i zmian w blokach ekstrakcji cech
YOLO26	C3k2, C2PSA	Detekcja end-to-end	Domyślna detekcja bez NMS oraz rezygnacja z DFL

informacji przestrzennych. To połączenie umożliwia modelowi prowadzenie detekcji na kilku skalach jednocześnie. Ostatnią częścią jest głowica detekcyjna (ang. *head*), która przewiduje klasy obiektów i położenie ich ramek ograniczających na podstawie dostarczonych z poprzedniej części map cech. Model YOLO26 wyposażony jest w dwie głowice detekcyjne: głowicę *one-to-many* i głowicę *one-to-one*. Pierwsza odpowiada za generowanie dużej liczby kandydatów obiektów, dostarczając gęstszy sygnał nadzorującego podczas uczenia, a druga uczy się przypisywać pojedynczą predykcję do każdego obiektu referencyjnego i jest domyślnie używana podczas inferencji [19].

Co ważne, podczas domyślnego wnioskowania model YOLO26 wykorzystuje głowicę *one-to-one*. Dzięki temu, że predykcja jest jednoznacznie przypisana, model może dokonywać detekcji bez użycia algorytmu NMS, a co za tym idzie może realizować detekcję typu end-to-end. W sytuacjach, w których użycie algorytmu NMS jest dopuszczalne model może korzystać z głowicy *one-to-many*, aby uzyskać większą skuteczność kosztem dodatkowego czasu [19].

Kolejnym ważnym aspektem wersji 26 jest fakt rezygnacji z użycia DFL (ang. *Distribution Focal Loss*) w trakcie regresji ramek, co pozwala zmniejszyć złożoność głowicy detekcyjnej i usunąć ograniczenia wynikające z dyskretnej reprezentacji odległości. Do

procesu uczenia wprowadzono mechanizm Progressive Loss, stopniowo zwiększający znaczenie głowicy, oraz STAL (ang. *Small-Target-Aware Label Assignment*), poprawiający przypisanie przykładów pozytywnych dla małych obiektów. W procesie optymalizacji wykorzystano optymalizator MuSGD [19].



Rysunek 2.2: Uproszczony schemat architektury detektora YOLO26 [19].

Na rysunku 2.2 przedstawiono uproszczony schemat współczesnej architektury modelu YOLO26. Ukazuje on znaczny rozwój względem pierwszej wersji modelu i wyraźne odejście od przekształcania wejścia w pojedynczą siatkę predykcji. Zastąpiono ją hierarchicznym przetwarzaniem cech, ich wieloskalowym połączeniem i dostosowaną głowicą decyzyjną. Pomimo tych zmian, zachowano główną ideę rodziny, czyli detekcję przez jeden model, bez osobnego etapu generowania propozycji regionów.

2.6.2 Warianty ze względu na realizowane zadanie

Dzisiejsza implementacja rodziny YOLO przez Ultralytics nie ogranicza się jedynie do klasycznej detekcji, a stara się dostarczać wielu modeli obsługujących różne zadania. Głównym kierunkiem jest wciąż model klasycznej detekcji, ale jego architektura może zostać rozszerzona o dodatkowe głowice przeznaczone do obsługi innych zadań. Obecnie dostępne modele mogą obsługiwać zadania detekcji, klasyfikacji, segmentacji instancji, segmentację semantyczną, estymację głębi, estymację pozy oraz detekcję z obróconymi ramkami [50]. Poszczególne zadania zostały zestawione w tabeli 2.3.

Ze względu na specyfikę systemu spośród dostępnych wariantów YOLO26 do dalszej analizy wybrano model przeznaczony do detekcji obiektów. Wynik zawierający klasę obiektu, wartość pewności oraz jego prostokątną ramkę ograniczającą dostarcza wystarczającej liczby informacji o obiekcie by móc zrealizować moduł detekcji. Segmentacja instancji umożliwiłaby dokładne wyodrębnianie obiektu z tła, ale wymagałaby przygotowania masek segmentacyjnych dla zbioru uczącego, a także zwiększyłaby złożoność

Tabela 2.3: Wybrane zadania obsługiwane przez modele YOLO26

Zadanie	Oznaczenie modelu	Wynik działania
Detekcja obiektów	yolo26*.pt	Klasy obiektów, wartości pewności i prostokątne ramki ograniczające
Segmentacja instancji	yolo26*-seg.pt	Klasy, ramki ograniczające i osobne maski pikselowe obiektów
Segmentacja semantyczna	yolo26*-sem.pt	Przypisanie klasy do każdego piksela obrazu
Estymacja głębi	yolo26*-depth.pt	Mapa przewidywanej głębi obrazu
Klasyfikacja obrazu	yolo26*-cls.pt	Klasa przypisana do całego obrazu
Estymacja pozy	yolo26*-pose.pt	Położenie punktów charakterystycznych obiektu lub sylwetki
Detekcja z ramkami obróconymi	yolo26*-obb.pt	Klasy i ramki ograniczające rozszerzone o orientację obiektu

obliczeniową modelu. Możliwym kierunkiem dalszego rozwoju może być wykorzystanie wariantu z ramkami obróconymi, które mogłyby ograniczyć udział tła w obszarze detekcji.

2.6.3 Warianty ze względu na skalę modelu

Każdy wariant YOLO26 jest dostępny w kilku skalach różniących się od siebie liczbą parametrów oraz kosztem obliczeniowym. Wybór odpowiedniego wariantu powinien być oparty na wymaganiach systemu oraz ograniczeniach zasobów sprzętowych urządzenia końcowego. Autorzy modelu detekcyjnego przewidzieli pięć wariantów: **n** (*nano*), **s** (*small*), **m** (*medium*), **l** (*large*) oraz **x** (*extra large*). W referencyjnych wynikach każdy kolejny wariant osiąga wyższe wartości *mAP*50-95, wymagając jednocześnie większej liczby parametrów i operacji [50].

Tabela 2.4: Porównanie wariantów detekcyjnych YOLO26 na zbiorze COCO dla obrazów 640 × 640. Opracowanie własne na podstawie [50].

Wariant	mAP50-95	Parametry [mln]	GFLOPs	T4 TensorRT10 [ms]
YOLO26n	40,9	2,4	5,4	1,7
YOLO26s	48,6	9,5	20,7	2,5
YOLO26m	53,1	20,4	68,2	4,7
YOLO26l	55,0	24,8	86,4	6,2
YOLO26x	57,5	55,7	193,9	11,8

Wyniki, które zostały przedstawione w tabeli 2.4 pokazują kompromis, typowy dla tej rodziny modeli. Wzrost rozmiaru modelu skorelowany jest ze wzrostem wartości mAP_{50-95} , liczby parametrów, liczby wykonywanych operacji, oraz czasem wykonania. Największy wzrost skuteczności pomiędzy sąsiednimi wariantami obserwujemy przy przejściu z modelu YOLO26n do YOLO26s: wartość mAP_{50-95} wzrasta o 7,7 punktu. Dalsze zwiększanie rozmiaru przynosi coraz mniejsze przyrosty względem rosnącego kosztu obliczeniowego.

Podane wartości czasowe służą wyłącznie porównaniu wariantów w jednakowych warunkach testowych i nie mogą być utożsamiane z wartością rzeczywistą czasu wykonania. Ten zależy od sprzętu, rozdzielczości obrazu wejściowego, sposobu wdrożenia i innych elementów implementacji.

2.6.4 Dobór wariantów modelu

Do dalszej analizy przyjęto najnowszą wersję modeli rozwijanych przez Ultralytics – YOLO26. Wprowadza ona rozwiązania istotne z punktu projektowanego systemu, przede wszystkim domyślną detekcję end-to-end bez etapu NMS i uproszczoną regresję ramek. Te zmiany ograniczają złożoność głowicy i zakres przetwarzania, jaki był wykonywany po zakończeniu pracy sieci [19].

Ze względu na wymaganie działania modelu w RT oraz założenie wykorzystania systemu w środowisku szkolnym, dalszą analizę należy skupić na najmniejszych wariantach modelu: **n** i **s**. Wynika to z konieczności uwzględnienia ograniczeń i heterogeniczności infrastruktury sprzętowej w polskich placówkach oświatowych. Zieliński w badaniu z 2023 roku dotyczącym cyfryzacji polskich szkół wskazuje na problem przestarzałego sprzętu komputerowego, często mającego ponad 5 lat użytkowania [64].

Wariant **n** charakteryzuje się najmniejszą liczbą parametrów i najniższym kosztem obliczeniowym, co czyni go naturalnym kandydatem dla urządzeń o ograniczonych zasobach. Wariant **s** daje natomiast istotny wzrost wartości mAP_{50-95} , dlatego może lepiej odpowiadać wymaganiom stawianym systemowi. Uwzględnienie obu wersji pozwoli przeanalizować wpływ poszczególnych wariantów i kombinacji parametrów na wydajność i dokładność detekcji.

2.7 Wykorzystanie danych wizualnych i przestrzennych w rozpoznawaniu obiektów

Rozpoznawanie obiektów nie musi opierać się jedynie na klasycznym obrazie RGB. Współczesne urządzenia mogą oferować również dostęp do danych opisujących geometrię obserwowanej sceny, przedstawionych w postaci chmur punktów czy map głębi. Dane

przestrzenne mogą bardzo dobrze uzupełniać informacje o kolorze, teksturze i strukturze obrazu rejestrowanego przez kamerę. Każdy z wymienionych sposobów reprezentacji danych wiąże się jednak z odmiennymi ograniczeniami i wymaganiami, które należy rozpatrywać w kontekście konkretnego zastosowania.

2.7.1 Obraz RGB

Obrazy kolorowe, najczęściej reprezentowane w przestrzeni RGB (ang. *red-green-blue*), stanowią obecnie jedną z najpowszechniejszych form reprezentacji danych w dziedzinie wizji komputerowej [55]. Kamery są obecnie wykorzystywane w większości dziedzin życia: monitoringu, medycynie, przemyśle, motoryzacji czy w urządzeniach osobistych. Istotne jest więc określenie, jakie informacje zawarte w zarejestrowanym obrazie mogą zostać wykorzystane do rozpoznawania obiektów 3D.

Obraz zarejestrowany przez kamerę jest reprezentacją trójwymiarowej sceny na płaszczyźnie dwuwymiarowej. Pomimo utraty bezpośredniej informacji o głębi, z pojedynczej klatki wyodrębnić można wiele informacji użytecznych podczas analizy obiektów, m.in. o kolorze, teksturze, krawędziach, zmianach jasności czy cieniach. Te informacje mogą z powodzeniem zostać wykorzystane przez modele uczenia maszynowego do wyznaczenia charakterystycznych cech przedstawionych obiektów. Obrazy RGB stanowią również podstawową formę danych wejściowych dla wielu CNN oraz detektorów obiektów, w tym rodziny YOLO opisanej w sekcji 2.6.

Rejestracja obrazu przez kamerę cyfrową jest oparta na procesie przejścia światła odbitego przez układ optyczny i skupieniu go na światłoczułym sensorze. Współczesne kamery cyfrowe do rejestracji obrazu wykorzystują głównie sensory CMOS (ang. *complementary metal-oxide-semiconductor*), a historycznie szeroko stosowano sensory CCD (ang. *charge-coupled device*). Dane zarejestrowane przez sensor są zwykle przetwarzane dalej w celu ich odszumienia, korekty kolorów czy poprawienia balansu bieli. Wynik może zostać zapisany w formacie skompresowanym JPEG (ang. *Joint Photographic Experts Group*) lub HEIF (ang. *High Efficiency Image File Format*) [47].

Po przetworzeniu obraz RGB może być reprezentowany w postaci macierzy trójwymiarowej o wymiarach

$$M \times N \times 3, \quad (2.8)$$

gdzie M i N to liczba wierszy i kolumn obrazu, a trzeci wymiar odpowiada kanałom barwnym: czerwonemu, zielonemu i niebieskiemu. Wartość pojedynczego piksela jest więc krotką trzech wartości, które można zapisać jako

$$I(i, j) = (R, G, B). \quad (2.9)$$

Pojedynczy piksel zawiera jedynie informację o wartościach trzech składowych koloru, ale zależności przestrzenne pomiędzy wieloma pikselami pozwalają odwzorować strukturę obserwowanej sceny. Lokalne zmiany wartości pikseli pozwalają m.in. wyznaczać gradienty obrazu, które mogą posłużyć do wykrywania krawędzi i innych strukturalnie ważnych elementów [46]. Jednakże, pomimo bogactwa informacji dostępnych do odczytu w samym obrazie, reprezentacja ta ma również istotne ograniczenia. Przede wszystkim, wygląd obserwowanego obiektu zależy znacząco od warunków oświetleniowych i stopnia zasłonięcia. Zmiana tych warunków może doprowadzić do różnic w kolorach, kontraście czy widoczności jego krawędzi, tym samym wpływając na skuteczność jego rozpoznawania [47]. Brak informacji o odległości punktu od kamery jest również istotnym ograniczeniem.

Projekcja dwuwymiarowa skutkuje utratą części informacji przestrzennej, przez co różne obiekty lub ich różne orientacje mogą prowadzić do podobnych obrazów. Dodatkowym utrudnieniem może być częściowe zasłonięcie obiektów. Istnieją modele estymujące głębię, np. YOLO26*-depth 2.6.2, opisany w sekcji 2.6.2, jednak otrzymane z nich wartości nie stanowią pomiaru odległości, a ich dokładność zależy od wielu czynników. Z tego względu uzupełnienie obrazu o informację dotyczącą odległości punktów od kamery wydaje się przydatne podczas procesu rozpoznania brył trójwymiarowych.

2.7.2 Pozyskiwanie informacji o głębi

Informacja dotycząca głębi opisuje położenie aktualnie obserwowanych punktów względem kamery. Jeżeli jej wartości przypiszemy do poszczególnych pikseli, otrzymujemy mapę głębi, którą możemy zapisać jako

$$D(i, j) = d, \quad (2.10)$$

gdzie d to głębokość sceny odpowiadająca pikselowi (i, j) . Dzisiejsza technika pozwala na określanie głębi na kilka sposobów.

Najprostsza ze względu na wymaganą infrastrukturę jest estymacja głębi z pojedynczego obrazu RGB. Określana jest ona jako *monocular depth estimation*, a jej działanie opiera się na wyznaczeniu struktury przestrzennej na podstawie cech obrazu. Przykładami tej techniki są: rozwiązanie zaproponowane przez Ranftla et al. oraz Monodepth2 opracowane przez zespół Godarda [14, 36]. Te metody umożliwiają uzyskanie mapy głębi bez stosowania dodatkowych urządzeń. Należy jednak pamiętać, że wynik stanowi jedynie estymację odległości, a nie jest jej pomiarem.

Kolejną metodą jest stereowizja, wykorzystująca dwa obrazy zarejestrowane z różnych położzeń. Znając wzajemne położenie kamer i analizując dysparcję, czyli różnicę położenia odpowiadających sobie punktów na obrazach, możliwe jest wyznaczenie informacji o głębi. Technologia stereoskopowa znalazła zastosowanie w urządzeniach konsumenckich. W 2010 roku firma Nintendo zaprezentowała nową konsolę wykorzystującą właśnie to rozwiązanie.

Konsola Nintendo 3DS była wyposażona w dwie kamery, które umożliwiały rejestrację zdjęć stereoskopowych [18]. To rozwiązanie wymaga jednak znajomości parametrów kamer i ich wzajemnego położenia, a także kalibracji układu.

Ciekawym podejściem do omawianego problemu jest aktywny pomiar odległości, w którym urządzenie emituje sygnał i wyznacza odległość na bazie zarejestrowanego sygnału powrotnego. Jednym z takich urządzeń jest LiDAR (ang. *Light Detection and Ranging*), który emituje promień lasera i analizuje sygnał odbity od obiektów. Jeżeli dane rozwiązanie wykorzystuje metodę czasu przelotu, to odległość jest wyznaczana na podstawie czasu między emisją impulsu a zarejestrowaniem jego powrotu [40].

2.7.3 Dane LiDAR i chmury punktów

Dane pozyskane z czujnika LiDAR mogą być wykorzystane do zbudowania reprezentacji przestrzennej sceny. Jako formę tej reprezentacji często stosuje się chmurę punktów (ang. *point cloud*), będącą zbiorem punktów rozmieszczonych w przestrzeni 3D. Można ją przedstawić jako zbiór

$$P = \{p_i\}_{i=1}^N, \quad (2.11)$$

gdzie każdy punkt p_i jest reprezentowany jako wektor

$$p_i = (x_i, y_i, z_i). \quad (2.12)$$

Zależnie od źródła danych, do poszczególnych punktów mogą zostać dodane dodatkowe informacje, np. intensywność sygnału, kolor lub wektor normalny. Chmura punktów, w przeciwieństwie do obrazu RGB, nie musi tworzyć regularnej siatki i zazwyczaj traktuje się ją jak nieuporządkowany zbiór punktów. Kolejność punktów w zbiorze nie powinna mieć znaczenia dla reprezentacji geometrii obiektu [4].

Brak regularnej struktury chmury punktów powoduje, że jej przetwarzanie nie może być zrealizowane w identyczny sposób jak obrazu. Z tego powodu zaproponowano wiele architektur przeznaczonych do bezpośredniego przetwarzania tego typu danych. Jedną z nich jest PointNet, który przetwarza cechy poszczególnych punktów, a następnie wykonuje ich symetryczną agregację. Dzięki temu wynik nie jest zależny od kolejności punktów wejściowych. Ta architektura jest wykorzystywana m.in. do klasyfikacji i segmentacji obiektów trójwymiarowych [4].

Podstawowa wersja PointNet w ograniczonym stopniu uwzględnia lokalne zależności między punktami, co poskutkowało opracowaniem jej rozwinięcia, PointNet++. Wprowadzono w nim hierarchiczne grupowanie punktów oraz uczenie cech dla kolejnych obszarów przestrzennych. Umożliwia to analizę geometrii dla różnych poziomów szczegółowości oraz lepsze uwzględnienie zmiennej gęstości próbek, która występuje w chmurach punktów [35].

Chmury punktów mogą być wykorzystywane bezpośrednio do detekcji obiektów, czego przykładem jest VoteNet. Jego architektura oparta jest na PointNet++ oraz mechanizmie głosowania Hougha. Kolejne punkty wraz z wyznaczonymi cechami generują głosy wskazujące środki potencjalnych obiektów. Głosy następnie są grupowane, a na ich podstawie wyznacza się trójwymiarowe ramki ograniczające i klasy. To rozwiązanie nie wymaga wiedzy o kolorze ani teksturze obiektów, a bazuje jedynie na chmurze, bez potrzeby konwertowania jej do regularnej reprezentacji przestrzennej [33].

Skuteczność rozpoznania obiektów w chmurze punktów zależy silnie od jakości danych wejściowych i metody ich analizy. Rzeczywiste pomiary mogą zawierać w sobie losowe szумы, odstające punkty, czy zmienną gęstość próbek. Ten problem uwidacznia się szczególnie przy analizie niewielkich elementów geometrii, które przy chmurze o małej gęstości mogą być reprezentowane przez pojedyncze punkty. Jak wskazują Zhang et al., takie zakłócenia obecne w rzeczywistych chmurach punktów mogą prowadzić do pogorszenia skuteczności rozpoznawania obiektów. Odpowiedzią autorów na ten problem jest architektura R-PointNet, która zwiększa odporność modelu na zakłócenia [63].

Same dane pochodzące z LiDAR są przede wszystkim opisem geometrii obserwowanej sceny i nie przenoszą informacji o kolorze ani teksturze powierzchni. Istnieją jednak metody łączenia danych z kamer z danymi przestrzennymi. Zalety jednej reprezentacji mogą częściowo kompensować ograniczenia drugiej. Z tego powodu możliwe jest jednoczesne wykorzystanie, w procesie detekcji.

2.7.4 Fuzja danych wizualnych i przestrzennych

Zależnie od etapu, na którym następuje fuzja danych, możliwe jest podzielenie jej na kilka poziomów: fuzję danych, fuzję cech i fuzję decyzji [24].

Połączenie ze sobą danych wizualnych i przestrzennych sprawia, że otrzymujemy wspólną reprezentację sceny. Obraz z kamery dostarcza informacji o kolorze i krawędziach, a mapy głębi i chmury punktów dają nam możliwość bezpośredniego umiejscowienia punktów w przestrzeni, ukazując geometrię przestrzenną całej sceny. Ograniczenia każdej z tych modalności stanowią motywację naukowców do opracowywania metod ich wspólnego użycia. W literaturze znajdujemy wiele metod fuzji danych. Celem tego działania jest uzyskanie reprezentacji, która będzie zawierała w sobie zarówno cechy wizualne, jak i przestrzenne obserwowanych obiektów. Takie podejście zastosowano w modelu ImVoteNet, będącym rozwinięciem architektury VoteNet. Model łączy ze sobą informacje z obrazów RGB z cechami pochodzącymi z chmury punktów. Na obrazie wyznaczane są cechy geometryczne, semantyczne i teksturowe, a następnie są one przenoszone do przestrzeni trójwymiarowej. Po połączeniu ich z cechami punktów następuje głosowanie, na wzór modelu VoteNet. Na zbiorze SUN RGB-D model osiągnął wartość $mAP@0.25$ wynoszącą 63,4%, poprawiając wynik swego poprzednika o 5,7 punktu [34].

Korzyści z łączenia danych wykazali również Liu et al. proponując model, który wykonywał równoległe detekcje na obrazie z kamery oraz mapie głębi z LiDAR. Ich rozwiązanie, łączące dane, uzyskało lepsze wyniki niż detekcje wykonywane osobno na obu modalnościach. Podczas eksperymentów model osiągnął wartość mAP równą 89,26%, podczas gdy warianty wykorzystujące pojedyncze źródło danych osiągnęły wartości 86,70% dla modelu korzystającego z obrazów i 74,27% dla modelu korzystającego z danych LiDAR. Jednocześnie wskazali, że przetworzenie pojedynczej klatki zajmowało jedynie 0,03 sekundy, co teoretycznie umożliwia działanie modelu w czasie rzeczywistym [24].

Architektury z rodziny YOLO mogą również stanowić element systemów wykorzystujących fuzję danych. Przykładami mogą być sieć YOLO-POINT, w której autorzy połączyli detektor YOLOv5 z siecią PointNet++, a także sieć DTC-YOLO, która wykorzystuje jednocześnie obraz RGB oraz dane LiDAR. W DTC-YOLO zastosowano odwzorowanie informacji o głębi na obraz i ważoną fuzję obu reprezentacji. Model wykorzystuje też sieć ADF³-Net, która stosuje dynamiczne bramkowanie do łączenia cech pochodzących z różnych poziomów. Eksperymenty z zastosowaniem modelu DTC-YOLO na zbiorze KITTI dały wynik mAP_{50} równy 0,911 oraz mAP_{50-95} równy 0,693, co przewyższa wyniki osiągnięte przez porównywane modele YOLOv8n, YOLOv10n oraz YOLO11n [27, 60].

Wskazuje to, że połączenie danych z różnorodnych modalności może poprawić skuteczność detekcji, a jednocześnie nie musi uniemożliwiać działania w czasie rzeczywistym. Jednym z ograniczeń, jakie należy wziąć pod uwagę, jest konieczność zastosowania dodatkowego sensora, jeżeli dane mają pochodzić z bezpośredniego pomiaru. Korzystanie z kilku źródeł danych może również wymagać ich kalibracji i synchronizacji. Ponadto przetwarzanie dodatkowego strumienia danych może zwiększyć znacząco wymagania obliczeniowe systemu. Przydatność fuzji należy więc rozpatrywać nie tylko z perspektywy potencjalnego wzrostu skuteczności detekcji, ale także z perspektywy dostępności odpowiednich sensorów, jakości pozyskiwanych danych oraz wymagań dotyczących czasu działania systemu.

2.8 Podsumowanie analizy tematu

Analiza literatury związanej z osobami niewidomymi uwidacznia kierunek, jaki należy przyjąć w trakcie projektowania systemu. Przede wszystkim należy skupić się na dostępności oprogramowania i wsparciu procesu nauki zarówno z perspektywy ucznia, jak i nauczyciela. Szczególną uwagę należy zwrócić na sposób przekazywania informacji użytkownikowi, tak by komunikaty były zrozumiałe, jednoznaczne i skutecznie wspierały proces nauki. Powinny również wspierać użytkownika w nauce, poprzez ułatwienie budowania reprezentacji przestrzennej danego obiektu. Dostępne systemy wspomagające często opierają się na komputerowym przetwarzaniu obrazu otoczenia i algorytmach detekcji obiektów, starając się działać w sposób uniwersalny – rozpoznając obiekty codziennego

użytku, jak przeszkody czy elementy otoczenia. Obszarem słabo zagospodarowanym pozostaje wsparcie niewidomych uczniów w nauce o geometrii przestrzennej. Możliwe rozwiązania obejmują systemy wizyjne, urządzenia wyposażone w mikrokontrolery czy różnego rodzaju sensory. Przy wyborze technologii należy jednak uwzględnić jej powszechność i koszt. Ze względu na ograniczone możliwości sprzętowe szkół uzasadnione jest wykorzystanie istniejącej infrastruktury, takiej jak telefony i komputery, bez konieczności zakupu wyspecjalizowanych urządzeń. Specyfika projektowanego systemu wprowadza dodatkowe utrudnienia związane z bezpośrednim kontaktem użytkownika z obiektem. Mimo że dłonie mogą zasłaniać część bryły, detekcja powinna zachować swoją skuteczność również w tych warunkach.

Przegląd możliwych danych wejściowych ujawnia problemy mogące wystąpić przy użyciu pojedynczego źródła danych. Obraz RGB z kamery daje informację o otoczeniu, kolorze i teksturze obiektów, zaś skanowanie przestrzenne pozwala odwzorować głębię i geometrię sceny, umożliwiając tworzenie map głębi lub chmur punktów. Analizowane prace wskazują, że fuzja danych z kamery i skanera LiDAR może prowadzić do poprawy skuteczności detekcji względem korzystania z pojedynczej modalności. Należy jednak przy tym zwrócić uwagę na ograniczenia takiego podejścia, jak konieczność kalibracji sensorów czy większe wymagania obliczeniowe. Z tego względu jako główne źródło danych przyjęto obrazy kolorowe, będące domyślnym wejściem sieci YOLO. Wykorzystanie danych o głębi oraz ich fuzja z obrazem RGB wymagają dodatkowej weryfikacji eksperymentalnej.

Porównanie możliwych metod detekcji wskazuje, że w tej sytuacji najodpowiedniejszym rozwiązaniem będzie użycie detektorów jednoetapowych z rodziny YOLO, w obrębie której zdecydowano się na użycie generacji YOLO26, ze względu na usprawnienie skuteczności i wydajności względem poprzednich wersji. Spośród dostępnych, wybrano warianty *n* i *s*, ze względu na korzystny kompromis pomiędzy skutecznością detekcji a kosztem obliczeniowym. Ich porównanie pozwoli ocenić, czy użycie modelu o większej liczbie parametrów wpłynie na poprawę wyników. Nie należy kategorycznie odrzucać pozostałych metod, jednak ich ograniczenia mogą znacznie wpłynąć na jakość i kulturę działania systemu.

Kolejne etapy pracy powinny obejmować określenie wymagań i wykonanie projektu systemu. Następnie należy przygotować zbiór danych, wyuczyć wybrane warianty sieci, porównać wyniki oraz zaimplementować mechanizm przekazywania informacji użytkownikowi.

Rozdział 3

Koncepcja systemu rozpoznawania dotykanych obiektów 3D

Przed przystąpieniem do implementacji systemu rozpoznawania dotykanych obiektów 3D, konieczne jest określenie jego koncepcji i wykonanie projektu. Analiza literatury naukowej pozwoliła nakreślić główne wymagania jakie spełniać powinien projektowany system. Najważniejszymi z nich są: rozpoznawanie dotykanych brył, również częściowo zasłoniętych, działanie w czasie zbliżonym do rzeczywistego, jednoznaczne, wspierające użytkownika informacje zwrotne i możliwość uruchomienia części obliczeniowej na powszechnie dostępnych komputerach, również tych o ograniczonych zasobach. Niniejszy rozdział przedstawia proponowaną koncepcję systemu, jego sposób działania, architekturę, sposób przepływu danych oraz najważniejsze decyzje projektowe.

System zaprojektowany jest w architekturze klient-serwer, gdzie klientem jest aplikacja mobilna, która rejestruje dane i przekazuje je do serwera uruchomionego na komputerze. Ten przetwarza dane, wykonuje detekcję obiektów i zwraca wynik. Wybór takiej architektury umożliwia użycie bardziej wymagających modeli niż byłoby to możliwe w przypadku uruchomienia ich na urządzeniu mobilnym.

3.1 Założenia funkcjonalne i нефункционалне

! Wróć do mnie po napisaniu rozdziału o koncepcji systemu, a potem implementacji, żeby zweryfikować!!!

Poprzez pogłębione studium literatury naukowej oraz analizę wymagań użytkowników określono główne założenia funkcjonalne i нефункционалне projektowanego systemu. Wynikają one z celu pracy, sposobów działania podobnych systemów oraz potrzeb, jakie zauważono przy analizie potrzeb użytkowników niewidomych.

Założenia funkcjonalne

Projekt systemu zakłada spełnienie następujących założeń funkcjonalnych.

- Rejestracja obrazu dłoni i dotykanego obiektu przy pomocy kamery urządzenia mobilnego.
- Przesyłanie zarejestrowanych klatek obrazu do części obliczeniowej systemu uruchomionej na komputerze.
- Wykrycie obiektu znajdującego się w dłoniach użytkownika i rozpoznanie jego klasy przy pomocy modelu detekcji obiektów.
- Udostępnienie użytkownikowi informacji zwrotnej o rozpoznanym obiekcie i aktualnie eksplorowanej ścianie w formie komunikatu głosowego.
- Dostępność interakcji i informacji zwrotnej.
- Wykrywanie obiektu przy zróżnicowanych ułożeniach dłoni i obiektu, w tym przy częściowym zasłonięciu obiektu.
- Obsługa sytuacji, gdy system nie rozpozna obiektu lub detekcja zakończy się wynikiem o zbyt niskiej pewności. Użytkownik powinien zostać poinformowany o tym fakcie i zachęcony do zmiany ułożenia dłoni lub obiektu.
- Aktywacja modułu głosowego bez odrywania dłoni od przedmiotu, poprzez komendę głosową.
- Detekcja działająca bez przerwy, umożliwiającą ciągłe rozpoznawanie obiektów.
- Otrzymanie informacji na żądanie użytkownika.

Założenia niefunkcjonalne

Założenia niefunkcjonalne projektowanego systemu skupiają się na zapewnieniu odpowiedniej jakości jego działania. Ich wyznaczenie ma skutkować wytworzeniem systemu, który będzie użyteczny i wygodny w codziennym użytkowaniu. Wyszczególniono następujące założenia niefunkcjonalne.

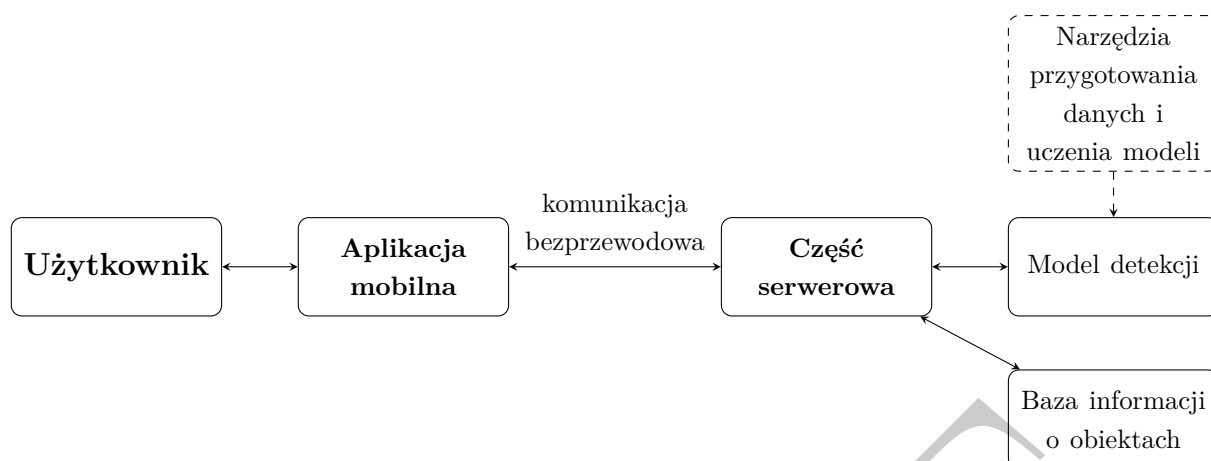
- Krótki czas odpowiedzi – wynik powinien być dostępny w jak najkrótszym czasie, zbliżonym do rzeczywistego.
- Działanie w warunkach obniżonej mocy obliczeniowej – część obliczeniowa systemu powinna móc być uruchomiona i działać na komputerach o ograniczonych zasobach, np. laptopach.

- Prostota obsługi – system nie powinien wymagać od użytkownika wykonywania skomplikowanych czynności, a jego obsługa nie powinna wymagać specjalistycznej wiedzy.
- Mobilność – urządzenie rejestrujące dane powinno dać się swobodnie przenosić i używać w różnych miejscach, np. w domu, w szkole czy w pracy i nie powinno wymagać specjalistycznego przygotowania miejsca pracy.
- Odporność na zmiany warunków – system powinien oferować porównywalną jakość działania w różnych warunkach oświetleniowych.
- Brak konieczności korzystania z specjalistycznego sprzętu – system powinien działać przy użyciu powszechnie dostępnych urządzeń, takich jak smartfony i komputery osobiste.
- Rozszerzalność – architektura systemu powinna umożliwiać wymianę części obliczeniowej na inną, np. w celu porównania różnych metod detekcji czy aktualizacji modelu.

3.2 Architektura systemu

Jako system rozpoznawania dotykanych obiektów 3D rozumie się zestaw powiązanych modułów, które współpracują ze sobą w celu realizacji wyznaczonych zadań. Architektura rozwiązania została zaprojektowana w taki sposób, aby możliwe było spełnienie wyznaczonych w sekcji 3.1 wymagań projektowych. Szczególna uwaga została poświęcona ograniczeniom wynikającym z użycia urządzeń mobilnych, krótkiemu czasowi odpowiedzi oraz możliwości obsługi systemu bez użycia wzroku. System został oparty na architekturze klient-serwer, w której odpowiedzialności zostały podzielone pomiędzy dwa główne moduły. Moduł kliencki, uruchamiany na urządzeniu mobilnym, odpowiada za rejestrację obrazu i obsługę interakcji z użytkownikiem. Część serwerowa, uruchamiana przez osobę obsługującą system na komputerze, realizuje zadania związane z analizą danych, detekcją obiektów i przygotowaniem informacji zwrotnej dla użytkownika. Komunikacja pomiędzy modułami odbywa się przez sieć bezprzewodową, umożliwiając przesyłanie danych oraz dwukierunkową wymianę komunikatów. Diagram blokowy architektury systemu przedstawiono na rysunku 3.1.

Aplikacja mobilna jest częścią systemu, z którą użytkownik wchodzi w bezpośrednią interakcję. Jej podstawowym zadaniem jest rejestracja obrazu obejmującego dłoń użytkownika oraz eksplorowany obiekt, a następnie przekazywanie go do części serwerowej systemu. Umożliwia ona również zainicjowanie żądania informacji o aktualnie eksplorowanym obiekcie. Po otrzymaniu z serwera komunikatu informacyjnego, aplikacja odczytuje go użytkownikowi za pomocą syntezy mowy. Część mobilna nie odpowiada więc za



Rysunek 3.1: Diagram blokowy architektury systemu

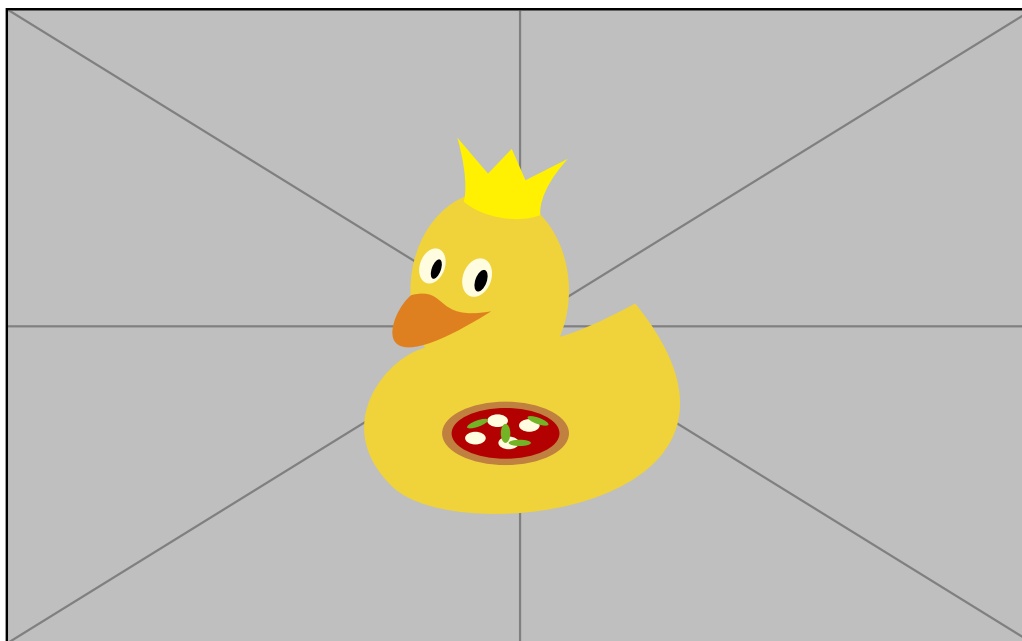
interpretację wyników ani wybieranie informacji, a pełni rolę pośrednika między użytkownikiem a częścią obliczeniową systemu.

Do odpowiedzialności serwera należą wszelkie zadania związane z przetwarzaniem obrazu i przygotowaniem informacji zwrotnej. W jego skład wchodzi moduły odpowiedzialne za komunikację z aplikacją, detekcję obiektów, interpretację wyników i generowanie komunikatów informacyjnych. Model detekcji analizuje przesłane klatki i określa najbardziej prawdopodobną klasę obiektu na obrazie wraz z wartością pewności predykcji. Następnie wynik detekcji jest udostępniany pozostałym modułom części serwerowej, które aktualizują informacje, przygotowują tekst na podstawie bazy informacji, a następnie przekazują go do odczytania w części klienckiej. Możliwe jest wielokrotne zapytanie o informacje o tym samym obiekcie bez konieczności zmiany jego położenia, ani orientacji względem kamery.

Projekt zakłada również utworzenie narzędzi służących do przygotowania danych treningowych i uczenia modeli detekcji. Są to elementy, które nie są niezbędne do działania systemu, ale pełnią istotną rolę w jego rozwoju. Wśród nich znajdują się skrypty tworzące zbiory danych treningowych z zarejestrowanych filmów, dzielące istniejące zbiory danych na podzbiory treningowe, walidacyjne i testowe oraz uczące modele detekcji. Dzięki nim możliwe jest również zestawienie wyników treningu umożliwiając ich porównanie i wybór najlepszego modelu.

3.3 Dobór klas rozpoznawanych obiektów

System przewiduje rozpoznanie sześciu klas obiektów, będących podstawowymi bryłami geometrycznymi: sześcianu, kuli, walca, prostopadłościanu, czworościanu i stożka. Przyjęto, że przyjęty zestaw klas jest wystarczający do realizacji założonego zakresu edukacyjnego oraz weryfikacji zaproponowanej koncepcji. Taki wybór klas umożliwia



Rysunek 3.2: Bryły geometryczne wykorzystywane w projektowanym systemie

użytkownikowi zapoznanie się z podstawowymi pojęciami związanymi z geometrią przestrzenną, jak ściana, powierzchnia boczna, krawędź czy wierzchołek. Bryły stanowią również podstawowe formy przestrzenne, z których możliwa jest budowa bardziej złożonych obiektów w wyniku ich łączenia, przekształcania i modyfikowania. Poznanie właściwości prostych brył może stanowić podstawę do rozumienia bardziej złożonych kształtów. Celem systemu nie jest więc pokrycie całego zakresu nauki stereometrii, lecz wsparcie użytkownika w nauce podstawowych pojęć i zależności geometrycznych. Znaczenie budowy skomplikowanych brył z prostych elementów ukazuje praca Figueiras i Arcavi [13] opisana w sekcji 2.1.

Na rysunku 3.2 przedstawiono fizyczne modele brył wykorzystanych do przygotowania danych treningowych. Ich powierzchnie zostały pomalowane przy użyciu stałej palety maksymalnie sześciu kolorów. Pozwala to na rozpoznanie przez system poszczególnych części bryły, a także przypisanie każdej z nich informacji, która ma być przekazana użytkownikowi. Kolor jednak nie definiuje bryły, a służy jedynie rozpoznaniu jej części. Mimo że poszczególne zestawy brył mogą mieć inne kolory od zestawu przedstawionego na rysunku, nie powinno to mieć wpływu na działanie systemu. W środowisku szkolnym mogą być wykorzystywane zestawy brył przygotowanych niezależnie, których kolorystyka może różnić się od zestawu użytego w pracach badawczych. Przez to model powinien rozpoznawać bryły na podstawie ich cech geometrycznych, a nie koloru.

Wybrane klasy obejmują obiekty o zróżnicowanej budowie, lecz posiadające wspólne cechy. Sześcian, prostopadłościan i czworościan posiadają ściany płaskie i wyraźne krawędzie, a kula, stożek oraz walec mają co najmniej jedną powierzchnię zakrzywioną. Te podobieństwa mogą skutkować podobnym wyglądem poszczególnych brył w zależności od

ich orientacji względem kamery. Predykcja może okazać się szczególnie trudna w przypadku, gdy obiekt jest w dużej części zasłonięty przez dłoń użytkownika. Uwzględnienie takich przypadków w zbiorze uczącym jest istotne dla uczenia modelu detekcji w trudnych warunkach. Pomyślne rozpoznanie częściowo zasłoniętego obiektu pozwala ocenić odporność modelu na zasłonięcia bryły.

Każda klasa odpowiada jednemu modelowi bryły, niezależnie od jej orientacji oraz aktualnie widocznej powierzchni. Kolor, orientacja i widoczna część obiektu nie stanowią osobnych klas, a są one analizowane dopiero po rozpoznaniu klasy przez model i zażądaniu informacji przez użytkownika. Po przeanalizowaniu wyników detekcji system przygotowuje odpowiedni komunikat informacyjny. Architektura systemu umożliwia późniejsze rozszerzenie zbioru rozpoznawalnych klas o kolejne obiekty po przygotowaniu dodatkowych danych treningowych.

3.4 Uzasadnienie wyboru metody detekcji

Dlaczego YOLO: szybkość, popularność, gotowe narzędzia treningowe, możliwość porównania wariantów n/s, metryki.

3.5 Przepływ danych w systemie

Od obrazu z telefonu, przez detekcję, po komunikat dla użytkownika.

3.6 Ograniczenia przyjętej koncepcji

Na przykład: kontrolowany zestaw obiektów, zależność od oświetlenia, ograniczony dataset, problem z generalizacją na inne dłonie/osoby/tła.

Rozdział 4

Implementacja systemu

Tu opisuję konkretną robotę inżynierską. Nie za dużo kodu w głównym tekście — raczej architektura, moduły, decyzje. Opis interfejsów aplikacji i środowisk projektowania lepiej dawać jako załącznik, bo wymagania sugerują, że takie rzeczy powinny iść do załącznika.

4.1 Środowisko programistyczne i wykorzystane narzędzia

Python, PyTorch/Ultralytics, OpenCV, Swift/iOS, ARKit, WebSocket, CVAT/Robo-flow/HF — tylko te, które finalnie zostaną.

4.2 Moduł akwizycji danych obrazowych

Jak pozyskuję obraz z telefonu / nagrań / streamu.

4.3 Moduł komunikacji klient-serwer

WebSocket, przesyłanie klatek, format danych, obsługa połączenia.

4.4 Moduł detekcji obiektów

Ładowanie modelu, inferencja, wynik detekcji, progi confidence.

4.5 Moduł informacji zwrotnej dla użytkownika

Komunikaty audio / tekstowe / docelowe zachowanie aplikacji.

4.6 Organizacja kodu i plików projektu

Krótko, bez robienia instrukcji obsługi. Dokładna dokumentacja może iść do załącznika albo plików dodatkowych.

DRAFT

Rozdział 5

Zbiór danych i metodyka badań

To jest bardzo ważny rozdział, bo wymagania kładą nacisk na metodykę, dane, powtarzalność i naukową analizę wyników. Badania powinny być przeprowadzone „tak jak jest przyjęte w ujęciu naukowym”, np. z zapewnieniem powtarzalności testów.

5.1 Charakterystyka zbioru danych

Liczba klas, liczba obrazów, rozdzielczość, źródło danych, przykładowe obrazy.

5.2 Proces pozyskiwania i oznaczania danych

Nagrania, ekstrakcja klatek, anotacje, negatywne próbki, puste etykiety.

5.3 Podział danych na zbiory treningowy, walidacyjny i testowy

Train/val/test. Tu trzeba bardzo jasno wyjaśnić, jak dzielić dane, żeby recenzent nie zarzucił przecieku danych.

5.4 Warianty danych wejściowych

Kolor, grayscale, ewentualne filtry/normalizacja. To będzie dobre miejsce na porównanie „kolor vs grayscale”.

5.5 Konfiguracja treningu modeli

Modele, rozmiar obrazu, batch, liczba epok, patience, pretrained/from scratch, sprzęt GPU.

5.6 Metryki oceny

Precision, recall, mAP50, mAP50-95, macierz pomyłek, czas inferencji/FPS, zużycie pamięci.

Do oceny modeli wykorzystano metryki zwracane przez narzędzie Ultralytics YOLO: precision, recall, mAP50 oraz mAP50-95. Dodatkowo na podstawie wartości precision i recall obliczono F1-score. Wyniki zapisywano w pliku results.csv, natomiast przebiegi metryk oraz macierze pomyłek analizowano na podstawie plików generowanych po zakończeniu treningu i walidacji. [49]

5.7 Procedura eksperymentalna

Czyli dokładnie: jakie modele trenowano, na jakich danych, jakie wyniki zbierano, jak porównywano.

Rozdział 6

Wyniki badań i ich analiza

To powinien być osobny rozdział, nie mieszany z metodyką. Wymagania mówią, że badania mają zawierać prezentację wyników, opracowanie i poszerzoną dyskusję wyników oraz wnioski.

6.1 Wyniki uczenia modeli YOLO

Tabele z najlepszymi wynikami: model, imgsz, precision, recall, mAP50, mAP50-95, f1, czas, pamięć.

6.2 Porównanie wariantów modeli

YOLO n vs s, ewentualnie inne warianty.

6.3 Wpływ reprezentacji obrazu na skuteczność detekcji

Kolor vs grayscale. To może być jeden z ciekawszych fragmentów pracy.

6.4 Analiza błędów klasyfikacji i detekcji

Macierze pomyłek, najczęściej mylone klasy, np. cube/sphere/cuboid itd.

6.5 Ocena działania systemu w scenariuszu użytkowym

Nie tylko mAP, ale też: czy system nadaje się do użycia, jakie ma opóźnienia, co działa, co nie działa.

6.6 Dyskusja ograniczeń wyników

Mały/średni dataset, kontrolowane warunki, jedna osoba w części danych, zależność od tła/oświetlenia, możliwe przeuczenie, generalizacja.

DRAFT

Rozdział 7

Podsumowanie i wnioski końcowe

7.1 Podsumowanie wykonanych prac

Co zostało zrobione: dataset, modele, system, eksperymenty.

7.2 Ocena realizacji celu pracy

Wprost: czy cel został osiągnięty, w jakim zakresie, z jakimi ograniczeniami.

7.3 Najważniejsze wnioski

Np. detekcja obiektów jest skuteczna, ale ma ograniczenia, np. w przypadku małych obiektów, podobnych kształtów, zależności od tła i oświetlenia.

7.4 Możliwe kierunki rozwoju

Większy dataset, więcej osób, więcej obiektów, integracja LiDAR/RGB-D, inference lokalnie na iPhone, lepszy feedback audio, testy z użytkownikami.

DRAFT

Bibliografia

- [1] Mirza Samad Ahmed Baig, Syeda Anshrah Gillani, Shahid Munir Shah, Mahmoud Aljawarneh, Abdul Akbar Khan i Muhammad Hamzah Siddiqui. *AI-based Wearable Vision Assistance System for the Visually Impaired: Integrating Real-Time Object Recognition and Contextual Understanding Using Large Vision-Language Models*. 2024. arXiv: 2412.20059 [cs.CV]. URL: <https://arxiv.org/abs/2412.20059>.
- [2] Fabiola Cando-Guanoluisa, Lesly Claudio, Dylan Cevallo, Carlos Colcha, Silvia Taípe i Grecia Pilatas. „Visually Impaired Students’ and Their Teacher’s Perceptions of the English Teaching and Learning Process”. W: *Mertesol Journal* 46 (sty. 2023), s. 1–14. DOI: 10.61871/mj.v46n4-3.
- [3] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov i Sergey Zagoruyko. *End-to-End Object Detection with Transformers*. 2020. arXiv: 2005.12872 [cs.CV]. URL: <https://arxiv.org/abs/2005.12872>.
- [4] R. Qi Charles, Hao Su, Mo Kaichun i Leonidas J. Guibas. „PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation”. W: *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2017, s. 77–85. DOI: 10.1109/CVPR.2017.16.
- [5] João Francisco Staffa Costa, V. M. R. Lima i M. S. Biembengut. „Spatial perception of a blind person: interactions and experiences”. W: *Brazilian Journal of Science Teaching and Technology, Ponta Grossa* 16 (2023), s. 1–20. DOI: 10.3895/rbect.v16n1.13944.
- [6] N. Dalal i B. Triggs. „Histograms of oriented gradients for human detection”. W: *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR’05)*. T. 1. 2005, 886–893 vol. 1. DOI: 10.1109/CVPR.2005.177.
- [7] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit i Neil Houlsby. *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. 2021. arXiv: 2010.11929 [cs.CV]. URL: <https://arxiv.org/abs/2010.11929>.

- [8] Ewa Dzierzgowska, Michał Maćkowski, Mateusz Kawulok, Piotr Brzoza, Stella Maćkowska i Dominik Spinczyk. „Alternative audio-graphic method for presenting structural information in mathematical graphs designed for low-vision users”. W: *Scientific Reports* 15.1 (2025), s. 21972. ISSN: 2045-2322. DOI: 10.1038/s41598-025-07710-2. URL: <https://doi.org/10.1038/s41598-025-07710-2>.
- [9] ETSI, CEN and CENELEC. *EN 301 549 V3.2.1: Accessibility Requirements for ICT Products and Services*. European Harmonised Standard. Mar. 2021. URL: https://www.etsi.org/deliver/etsi_en/301500_301599/301549/03.02.01_60/en_301549v030201p.pdf (term. wiz. 11.05.2026).
- [10] European Parliament and Council of the European Union. *Directive (EU) 2016/2102 of the European Parliament and of the Council of 26 October 2016 on the accessibility of the websites and mobile applications of public sector bodies*. Official Journal of the European Union, L 327, 2 December 2016, pp. 1–15. 2016. URL: <https://eur-lex.europa.eu/eli/dir/2016/2102/oj> (term. wiz. 11.05.2026).
- [11] European Parliament and Council of the European Union. *Directive (EU) 2019/882 of the European Parliament and of the Council of 17 April 2019 on the Accessibility Requirements for Products and Services*. Official Journal of the European Union, L 151, 7 June 2019, pp. 70–115. 2019. URL: <https://eur-lex.europa.eu/eli/dir/2019/882/oj> (term. wiz. 11.05.2026).
- [12] Mark Everingham, Luc Van Gool, Christopher K. I. Williams, John Winn i Andrew Zisserman. „The PASCAL Visual Object Classes (VOC) Challenge.” W: *International Journal of Computer Vision* 88.2 (2010), s. 303–338. DOI: 10.1007/s11263-009-0275-4.
- [13] Lourdes Figueiras i Abraham Arcavi. „Learning to See: The Viewpoint of the Blind”. W: *Selected Regular Lectures from the 12th International Congress on Mathematical Education*. Red. Sung Je Cho. Cham: Springer International Publishing, 2015, s. 175–186. ISBN: 978-3-319-17187-6. DOI: 10.1007/978-3-319-17187-6_10.
- [14] Clement Godard, Oisín Mac Aodha, Michael Firman i Gabriel Brostow. „Digging Into Self-Supervised Monocular Depth Estimation”. W: *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*. 2019, s. 3827–3837. DOI: 10.1109/ICCV.2019.00393.
- [15] Maşide Güler i Mustafa Ürey. „Are we aware of the needs of students with visual impairment? A study on challenges and instructional needs in science lessons”. W: *Journal of Pedagogical Sociology and Psychology* 7 (2 2025), s. 53–70. DOI: 10.33902/jpsp.202529253.
- [16] Kaiming He, Georgia Gkioxari, Piotr Dollár i Ross Girshick. *Mask R-CNN*. 2018. arXiv: 1703.06870 [cs.CV]. URL: <https://arxiv.org/abs/1703.06870>.

- [17] Intel Corporation. *New Intel-Created System Offers Professor Stephen Hawking Ability to Better Communicate with the World*. 2 grud. 2014. URL: <https://www.intc.com/news-events/press-releases/detail/380/new-intel-created-system-offers-professor-stephen-hawking> (term. wiz. 10.05.2026).
- [18] Yuichiro Ito i Yuki Nishimura. „Storage Medium Having Stored Therein Stereoscopic Image Display Program, Stereoscopic Image Display Device, Stereoscopic Image Display System, and Stereoscopic Image Display Method”. European Patent Application EP2395763A3. Ltd. Nintendo Co. i Inc. HAL Laboratory. 29 lut. 2012. URL: <https://patents.google.com/patent/EP2395763A3/en>.
- [19] Glenn Jocher, Jing Qiu, Mengyu Liu, Shuai Lyu, Fatih Cagatay Akyon i Muhammet Kalfaoglu. *Ultralytics YOLO26: Unified Real-Time End-to-End Vision Models*. Czer. 2026. DOI: 10.48550/arXiv.2606.03748.
- [20] Franklin Mingzhe Li, Lotus Zhang, Maryam Bandukda, Abigale Stangl, Kristen Shinohara, Leah Findlater i Patrick Carrington. „Understanding Visual Arts Experiences of Blind People”. W: *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. CHI '23. Hamburg, Germany: Association for Computing Machinery, 2023. ISBN: 9781450394215. DOI: 10.1145/3544548.3580941.
- [21] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He i Piotr Dollár. *Focal Loss for Dense Object Detection*. 2018. arXiv: 1708.02002 [cs.CV]. URL: <https://arxiv.org/abs/1708.02002>.
- [22] Tsung-Yi Lin, Michael Maire, Serge J. Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár i C. Lawrence Zitnick. „Microsoft COCO: Common Objects in Context”. W: *Computer Vision – ECCV 2014*. T. 8693. Lecture Notes in Computer Science. Springer, 2014, s. 740–755. DOI: 10.1007/978-3-319-10602-1_48.
- [23] John G. Linvill. „Reading Aid for the Blind”. Pat. US 3,229,387 (United States). 18 sty. 1966. URL: <https://patents.google.com/patent/US3229387A/en>.
- [24] Haibin Liu, Chao Wu i Huanjie Wang. „Real time object detection using LiDAR and camera fusion for autonomous driving”. W: *Scientific Reports* 13.1 (2023), s. 8056. ISSN: 2045-2322. DOI: 10.1038/s41598-023-35170-z. URL: <https://doi.org/10.1038/s41598-023-35170-z>.
- [25] Wei Liu, Dragomir Anguelov, Dumitru Erhan, Christian Szegedy, Scott Reed, Cheng-Yang Fu i Alexander C. Berg. „SSD: Single Shot MultiBox Detector”. W: *Computer Vision – ECCV 2016*. Springer International Publishing, 2016, s. 21–37. ISBN: 9783319464480. DOI: 10.1007/978-3-319-46448-0_2. URL: http://dx.doi.org/10.1007/978-3-319-46448-0_2.

- [26] David G. Lowe. „Distinctive Image Features from Scale-Invariant Keypoints”. W: *International Journal of Computer Vision* 60.2 (2004), s. 91–110. ISSN: 1573-1405. DOI: 10.1023/B:VISI.0000029664.99615.94. URL: <https://doi.org/10.1023/B:VISI.0000029664.99615.94>.
- [27] Zhengyi Ma, Jiayin Zhu, Jiayi Lu, Zhenliang Li, Xihan Cao, Jingyi Yao, Runjiu Hu i Genwang Liu. „Yolo-Point:Lidar-Image Fusion based on yolov5 and Pointnet++ for 3D Objection Perception”. W: sty. 2024.
- [28] R. G. Maling i D. C. Clarkson. „Electronic controls for the tetraplegic (possum) (patient operated selector mechanisms—P.O. S.M.)” W: *Spinal Cord* 1.3 (1963), s. 161–174. ISSN: 1476-5624. DOI: 10.1038/sc.1963.15. URL: <https://doi.org/10.1038/sc.1963.15>.
- [29] Mara Mills. „Hearing Aids and the History of Electronics Miniaturization”. W: *IEEE Annals of the History of Computing* 33.2 (2011), s. 24–45. DOI: 10.1109/MAHC.2011.43.
- [30] Rafael Padilla, Wesley L. Passos, Thadeu L. B. Dias, Sergio L. Netto i Eduardo A. B. da Silva. „A Comparative Analysis of Object Detection Metrics with a Companion Open-Source Toolkit”. W: *Electronics* 10.3 (2021). ISSN: 2079-9292. DOI: 10.3390/electronics10030279.
- [31] Kedar Potdar, Chinmay Pai i Sukrut Akolkar. „A Convolutional Neural Network based Live Object Recognition System as Blind Aid”. W: (list. 2018). DOI: 10.48550/arXiv.1811.10399.
- [32] Abhinav Pratap, Sushant Kumar i Suchinton Chakravarty. *Adaptive Object Detection for Indoor Navigation Assistance: A Performance Evaluation of Real-Time Algorithms*. 2025. arXiv: 2501.18444 [cs.CV]. URL: <https://arxiv.org/abs/2501.18444>.
- [33] Charles Qi, Or Litany, Kaiming He i Leonidas Guibas. „Deep Hough Voting for 3D Object Detection in Point Clouds”. W: paż. 2019, s. 9276–9285. DOI: 10.1109/ICCV.2019.00937.
- [34] Charles R. Qi, Xinlei Chen, Or Litany i Leonidas J. Guibas. „ImVoteNet: Boosting 3D Object Detection in Point Clouds With Image Votes”. W: *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2020, s. 4403–4412. DOI: 10.1109/CVPR42600.2020.00446.
- [35] Charles R. Qi, Li Yi, Hao Su i Leonidas J. Guibas. „PointNet++: deep hierarchical feature learning on point sets in a metric space”. W: *Proceedings of the 31st International Conference on Neural Information Processing Systems. NIPS’17*. Long Beach, California, USA: Curran Associates Inc., 2017, s. 5105–5114. ISBN: 9781510860964.

- [36] Rene Ranftl, Katrin Lasinger, David Hafner, Konrad Schindler i Vladlen Koltun. „Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer”. W: *IEEE Transactions on Pattern Analysis & Machine Intelligence* 44.03 (mar. 2022), s. 1623–1637. ISSN: 1939-3539. DOI: 10.1109/TPAMI.2020.3019967. URL: <https://doi.ieeecomputersociety.org/10.1109/TPAMI.2020.3019967>.
- [37] Joseph Redmon, Santosh Divvala, Ross Girshick i Ali Farhadi. *You Only Look Once: Unified, Real-Time Object Detection*. 2016. arXiv: 1506.02640 [cs.CV]. URL: <https://arxiv.org/abs/1506.02640>.
- [38] Joseph Redmon i Ali Farhadi. *YOLOv3: An Incremental Improvement*. 2018. arXiv: 1804.02767 [cs.CV]. URL: <https://arxiv.org/abs/1804.02767>.
- [39] Shaoqing Ren, Kaiming He, Ross Girshick i Jian Sun. *Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks*. 2016. arXiv: 1506.01497 [cs.CV]. URL: <https://arxiv.org/abs/1506.01497>.
- [40] Santiago Royo i Maria Ballesta-Garcia. „An Overview of LiDAR Imaging Systems for Autonomous Vehicles”. W: *Applied Sciences* 9.19 (2019), s. 4093. DOI: 10.3390/app9194093. URL: <https://doi.org/10.3390/app9194093>.
- [41] Rzeczpospolita Polska. *Ustawa z dnia 26 kwietnia 2024 r. o zapewnianiu spełniania wymagań dostępności niektórych produktów i usług przez podmioty gospodarcze*. Dz.U. z 2024 r. poz. 731. 2024. URL: <https://isap.sejm.gov.pl/isap.nsf/DocDetails.xsp?id=WDU20240000731> (term. wiz. 11.05.2026).
- [42] Rzeczpospolita Polska. *Ustawa z dnia 4 kwietnia 2019 r. o dostępności cyfrowej stron internetowych i aplikacji mobilnych podmiotów publicznych*. Dz.U. z 2019 r. poz. 848; tekst jednolity: Dz.U. z 2023 r. poz. 1440, z późn. zm. 2019. URL: <https://isap.sejm.gov.pl/isap.nsf/DocDetails.xsp?id=WDU20190000848> (term. wiz. 11.05.2026).
- [43] Pierre Sermanet, David Eigen, Xiang Zhang, Michael Mathieu, Rob Fergus i Yann LeCun. *OverFeat: Integrated Recognition, Localization and Detection using Convolutional Networks*. 2014. arXiv: 1312.6229 [cs.CV]. URL: <https://arxiv.org/abs/1312.6229>.
- [44] Sibi C. Sethuraman, Gaurav R. Tadkapally, Saraju P. Mohanty, Gautam Galada i Anitha Subramanian. *MagicEye: An Intelligent Wearable Towards Independent Living of Visually Impaired*. 2023. arXiv: 2303.13863 [cs.HC]. URL: <https://arxiv.org/abs/2303.13863>.
- [45] Dandan Shan, Jiaqi Geng, Michelle Shu i David F. Fouhey. „Understanding Human Hands in Contact at Internet Scale”. W: 2020. arXiv: 2006.06669 [cs.CV]. URL: <https://arxiv.org/abs/2006.06669>.

- [46] Irwin Sobel. „An Isotropic 3x3 Image Gradient Operator”. W: *Presentation at Stanford A.I. Project 1968* (lut. 2014).
- [47] Richard Szeliski. *Computer Vision: Algorithms and Applications*. 2nd. Texts in Computer Science. Springer Cham, 2022. DOI: 10.1007/978-3-030-34372-9. URL: <https://szeliski.org/Book/>.
- [48] Bugra Tekin, Federica Bogo i Marc Pollefeys. „H+O: Unified Egocentric Recognition of 3D Hand-Object Poses and Interactions”. W: *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2019, s. 4506–4515. DOI: 10.1109/CVPR.2019.00464.
- [49] Ultralytics. *Performance Metrics Deep Dive*. YOLO Performance Metrics documentation. 12 list. 2023. URL: <https://docs.ultralytics.com/guides/yolo-performance-metrics/> (term. wiz. 17.12.2025).
- [50] Ultralytics. *Ultralytics YOLO26*. 25 wrz. 2025. URL: <https://docs.ultralytics.com/models/yolo26> (term. wiz. 04.11.2025).
- [51] Ultralytics. *YOLO Architecture Explained: From YOLOv3 to YOLO26*. 6 lip. 2026. URL: <https://docs.ultralytics.com/guides/yolo-architecture> (term. wiz. 10.07.2026).
- [52] United Nations. *Convention on the Rights of Persons with Disabilities*. United Nations General Assembly Resolution A/RES/61/106. 2006. URL: <https://www.ohchr.org/en/instruments-mechanisms/instruments/convention-rights-persons-disabilities> (term. wiz. 11.05.2026).
- [53] Gregg Vanderheiden. „A journey through early augmentative communication and computer access”. W: *Journal of rehabilitation research and development* 39 (sty. 2003), s. 39–54.
- [54] P. Viola i M. Jones. „Rapid object detection using a boosted cascade of simple features”. W: *Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition. CVPR 2001*. T. 1. 2001, s. I–I. DOI: 10.1109/CVPR.2001.990517.
- [55] Saisai Wang, Jiashuai Cui, Fan Li i Liejun Wang. „Image Sampling Based on Dominant Color Component for Computer Vision”. W: *Electronics* 12.15 (2023), s. 3360. ISSN: 2079-9292. DOI: 10.3390/electronics12153360.
- [56] Wei Wang, Bin Jing, Xiaoru Yu, Yan Sun, Liping Yang i Chunliang Wang. „YOLO-OD: Obstacle Detection for Visually Impaired Navigation Assistance”. W: *Sensors* 24.23 (2024). ISSN: 1424-8220. DOI: 10.3390/s24237621. URL: <https://www.mdpi.com/1424-8220/24/23/7621>.

- [57] World Wide Web Consortium. *Web Content Accessibility Guidelines 1.0*. W3C Recommendation. Maj 1999. URL: <https://www.w3.org/TR/WAI-WEBCONTENT/> (term. wiz. 11.05.2026).
- [58] World Wide Web Consortium. *Web Content Accessibility Guidelines (WCAG) 2.2*. W3C Recommendation. 2024. URL: <https://www.w3.org/TR/WCAG22/> (term. wiz. 11.05.2026).
- [59] Jingxi Xu, Han Lin, Shuran Song i Matei Ciocarlie. *TANDEM3D: Active Tactile Exploration for 3D Object Recognition*. 2023. DOI: 10.48550/arXiv.2209.08772. arXiv: 2209.08772 [cs.CV]. URL: <https://arxiv.org/abs/2209.08772>.
- [60] W. Xu, X. Du, R. Li i L. Xing. „DTC-YOLO: Multimodal Object Detection via Depth-Texture Coupling and Dynamic Gating Optimization”. W: *Sensors* 25.18 (2025), s. 5731. DOI: 10.3390/s25185731.
- [61] Chenyangguang Zhang, Guanlong Jiao, Yan Di, Gu Wang, Ziqin Huang, Ruida Zhang, Fabian Manhardt, Bowen Fu, Federico Tombari i Xiangyang Ji. „MOHO: Learning Single-view Hand-held Object Reconstruction with Multi-view Occlusion-Aware Supervision”. W: 2024. arXiv: 2310.11696 [cs.CV]. URL: <https://arxiv.org/abs/2310.11696>.
- [62] Xuefeng ZHANG, Shaojie ZHANG, Xin CHEN i Jinhua ZHANG. „A tactile glove for object recognition based on palmar pressure and joint bending strain sensing”. W: *Journal of Measurement Science and Instrumentation* 16.2 (2025), s. 173–185. DOI: 10.62756/jmsi.1674-8042.2025017. URL: <https://www.sciopen.com/article/10.62756/jmsi.1674-8042.2025017>.
- [63] Zhongyuan Zhang, Lichen Lin i Xiaoli Zhi. „R-PointNet: Robust 3D Object Recognition Network for Real-World Point Clouds Corruption”. W: *Applied Sciences* 14 (kw. 2024), s. 3649. DOI: 10.3390/app14093649.
- [64] Robert Zieliński. „Cyfrowa edukacja dziś:między szansą a wyzwaniem – studium porównawcze”. W: *EBIS* 2.80 (2023), s. 234–267. ISSN: 1643-8779. DOI: 10.24131/3247.230212.
- [65] Zhengxia Zou, Keyan Chen, Zhenwei Shi, Yuhong Guo i Jieping Ye. „Object Detection in 20 Years: A Survey”. W: *Proceedings of the IEEE* 111.3 (2023), s. 257–276. DOI: 10.1109/JPROC.2023.3238524.

DRAFT

Dodatki

DRAFT

Dokumentacja techniczna

DRAFT

DRAFT

Spis skrótów i symboli

- IoU Intersection over Union – miara stopnia pokrycia ramki przewidywanej przez model z ramką referencyjną,
- TP True Positive – prawdziwy pozytywny, poprawnie wykryty obiekt,
- FP False Positive – fałszywy pozytywny, błędna detekcja zwrócona przez model,
- FN False Negative – fałszywy negatywny, obiekt rzeczywisty niewykryty przez model,
- PR Precision-Recall – zależność między precyzją a czułością,
- AP Average Precision – średnia precyzja wyznaczana na podstawie krzywej precision-recall,
- mAP mean Average Precision – średnia wartość AP obliczana dla wszystkich klas,
- F1 F1-score – średnia harmoniczna precyzji i czułości,
- VOC Visual Object Classes – zbiór nazw benchmarku PASCAL VOC wykorzystywanego w ocenie detekcji obiektów,
- COCO Common Objects in Context – nazwa zbioru i benchmarku Microsoft COCO,
- W3C World Wide Web Consortium – organizacja zajmująca się standaryzacją technologii internetowych,
- WCAG Web Content Accessibility Guidelines – wytyczne dotyczące dostępności treści internetowych,
- CNN Convolutional Neural Network – konwolucyjna sieć neuronowa,
- HOG Histogram of Oriented Gradients – histogramy gradientów kierunkowych,
- SIFT Scale-Invariant Feature Transform – transformacja cech niezależnych,
- RPN Region Proposal Network – sieć generująca propozycje regionów,
- YOLO You Only Look Once – rodzina modeli detekcji jednoetapowej obiektów w czasie rzeczywistym,

DETR DETection TRansformer – model detekcji obiektów wykorzystujący architekturę transformera,

NMS Non-Maximum Suppression – algorytm usuwania nakładających się ramek detekcji,

RT Real-Time – czas rzeczywisty,

SSD Single Shot MultiBox Detector – model detekcji jednoetapowej obiektów w czasie rzeczywistym,

FPS Frames Per Second – liczba kadrów na sekundę,

DFL Distribution Focal Loss – funkcja straty wykorzystywana w modelach detekcji obiektów,

RGB Red-Green-Blue – przestrzeń kolorów reprezentowana przez trzy kanały: czerwony, zielony i niebieski,

ONZ Organizacja Narodów Zjednoczonych – organizacja międzynarodowa zrzeszająca państwa świata,

GPS Global Positioning System – globalny system pozycjonowania satelitarnego,

3D Three-Dimensional – trójwymiarowy sposób reprezentacji obiektów lub przestrzeni,

YCB Yale–Carnegie Mellon University–Berkeley – zbiór obiektów i modeli wykorzystywany w badaniach nad manipulacją robotyczną,

SVM Support Vector Machine – maszyna wektorów nośnych wykorzystywana do klasyfikacji danych,

R-CNN Region-based Convolutional Neural Network – konwolucyjna sieć neuronowa analizująca proponowane regiony obrazu,

ECCV European Conference on Computer Vision – europejska konferencja naukowa dotycząca widzenia komputerowego,

AI Artificial Intelligence – sztuczna inteligencja,

*mAP*50 mean Average Precision at IoU 0.50 – średnia wartość AP wyznaczana przy progu IoU równym 0,5,

*mAP*50-95 mean Average Precision at IoU 0.50–0.95 – średnia wartość AP obliczana dla progów IoU od 0,5 do 0,95,

FPN Feature Pyramid Network – sieć piramidy cech umożliwiającą analizę obiektów w wielu skalach,

- CSP Cross Stage Partial – mechanizm częściowego podziału i ponownego łączenia map cech,
- C3 CSP Bottleneck with 3 Convolutions – blok sieci wykorzystujący mechanizm CSP i trzy operacje konwolucji,
- PAN Path Aggregation Network – sieć agregacji ścieżek służąca do wieloskalowego łączenia cech,
- SPPF Spatial Pyramid Pooling – Fast – szybki moduł przestrzennego grupowania piramidowego,
- C2f CSP Bottleneck with two convolutions and feature fusion – blok CSP wykorzystujący dwie operacje konwolucji i fuzję cech,
- C3k2 C3k2 block – wariant bloku CSP wykorzystujący moduły C3k lub bloki typu bottleneck,
- C2PSA Cross-Stage Partial Spatial Attention – blok CSP wykorzystujący mechanizm uwagi przestrzennej,
- STAL Small-Target-Aware Label Assignment – mechanizm przypisywania etykiet uwzględniający małe obiekty,
- MuSGD Muon-inspired Stochastic Gradient Descent – hybrydowy optymalizator łączący SGD z rozwiązaniami inspirowanymi optymalizatorem Muon,
- GFLOPs Giga Floating-Point Operations – liczba miliardów operacji zmiennoprzecinkowych,
- CMOS Complementary Metal-Oxide-Semiconductor – technologia wykonywania światłoczułych sensorów obrazu,
- CCD Charge-Coupled Device – rodzaj światłoczułego sensora obrazu wykorzystującego elementy ze sprzężeniem ładunkowym,
- JPEG Joint Photographic Experts Group – stratny format kompresji i zapisu obrazów,
- HEIF High Efficiency Image File Format – wysokowydajny format zapisu obrazów,
- LiDAR Light Detection and Ranging – technologia pomiaru odległości za pomocą światła laserowego,
- RGB-D Red-Green-Blue and Depth – reprezentacja danych zawierająca obraz kolorowy i informację o głębi,
- SUN Scene Understanding – nazwa zbioru danych przeznaczonego do badań nad rozumieniem scen,

KITTI Karlsruhe Institute of Technology and Toyota Technological Institute – zbiór danych i zestaw benchmarków wykorzystywany w badaniach nad autonomiczną jazdą i widzeniem komputerowym,

YOLO-OD You Only Look Once–Obstacle Detection – model detekcji przeszkód przeznaczony do wspomagania nawigacji osób z niepełnosprawnością wzroku,

YOLO-POINT YOLO-POINT – model łączący detekcję YOLO z informacjami pochodzącymi z obrazu i chmury punktów,

DTC-YOLO Depth-Texture Coupling Mechanism YOLO – multimodalny model detekcji wykorzystujący połączenie informacji o głębi i teksturze,

ADF³-Net Adaptive Dimension-aware Focused Fusion Network – sieć adaptacyjnej fuzji cech pochodzących z różnych wymiarów i poziomów, ““

Lista dodatkowych plików, uzupełniających tekst pracy (jeżeli dotyczy)

W systemie do pracy dołączono dodatkowe pliki zawierające:

- źródła programu,
- zbiory danych użyte w eksperymentach,
- film pokazujący działanie opracowanego oprogramowania lub zaprojektowanego i wykonanego urządzenia,
- itp.

DRAFT

Spis rysunków

2.1	Uproszczony schemat architektury pierwszego modelu YOLO [37].	20
2.2	Uproszczony schemat architektury detektora YOLO26 [19].	22
3.1	Diagram blokowy architektury systemu	34
3.2	Bryły geometryczne wykorzystywane w projektowanym systemie	35

DRAFT

Spis tabel

2.1	Porównanie wybranych metod detekcji obiektów	16
2.2	Wybrane etapy rozwoju architektury modeli YOLO. Opracowanie własne na podstawie [19, 37, 38, 51].	21
2.3	Wybrane zadania obsługiwane przez modele YOLO26	23
2.4	Porównanie wariantów detekcyjnych YOLO26 na zbiorze COCO dla obra- zów 640×640 . Opracowanie własne na podstawie [50].	23